

Design, Verification, and ASIC Implementation of a Low-Power Multi-Clock Digital Communication SoC Subsystem

100 MHz Dual-Clock Domain SoC Architecture with Robust Clock Domain Crossing, Integrated Clock Gating, and Complete TSMC 130nm ASIC Backend Flow

PROJECT SPECIFICATION

Architecture:	Dual-Clock Domain SoC
System Clocks:	REF_CLK (100 MHz) UART_CLK (3.6864 MHz)
Technology:	TSMC 130nm CMOS
Cell Library:	scmetro_tsmc_c1013g
EDA Tools:	Synopsys DC, DFT, Formality, SpyGlass, QuestaSim/ModelSim
Repository:	GitHub Source Code

DESIGNER & ACADEMIC DETAILS

Designer:	Ahmed Badawy
Affiliation:	Cairo University Faculty of Engineering
Email:	ahmed.badwy05@eng-st.cu.edu.eg
GitHub:	@ahmedbadwy77
Specialization:	Digital IC Design & Verification
Date:	Summer 2026

Digital Design Diploma Project

RTL Design • Self-Checking Verification • Synthesis • DFT • Formal Equivalence • SpyGlass CDC • Place &
Route • GDSII

Faculty of Engineering, Cairo University • Academic Year 2025–2026

Contents

Abbreviations	5
1 Introduction and Project Scope	6
2 Multi-Clock System Architecture and Clock Domain Partitioning	8
2.1 Physical Clock Domain Partitioning	8
3 Comprehensive Clock Domain Crossing (CDC) Synchronization	9
3.1 Theoretical Foundations: Metastability and Multi-Bit Skew Hazards	9
3.2 Multi-Bit Data Synchronizer (DATA_SYNC)	10
3.2.1 Architectural Function & Mechanics	10
3.2.2 Design Rationale: Why We Used It (Problem Solved)	10
3.2.3 Key Engineering Benefits	11
3.3 Dual-Clock Asynchronous FIFO (ASYNC_FIFO)	11
3.3.1 Architectural Function & Mechanics	11
3.3.2 Design Rationale: Why We Used It (Problem Solved)	12
3.3.3 Why Gray-Coded Pointers Are Mandatory	12
3.3.4 Cummings' Full and Empty Detection Formulation	13
3.4 Dual-Domain Asynchronous Reset Synchronizers (RST_SYNC)	13
3.4.1 Architectural Function & Mechanics	13
3.4.2 Design Rationale: Why We Used It (Problem Solved)	14
4 Master Communication Protocol and Frame Formats	14
4.1 UART Serial Frame Specification	14
4.2 Baud Rate and Oversampling Calculations	15
4.3 Master System Command Set	15
5 Detailed Subsystem Hardware Architecture	16
5.1 System Controller (SYS_CTRL)	16
5.1.1 Architectural Function & Mechanics	16
5.1.2 Design Rationale: Why We Used It (Problem Solved)	16
5.1.3 Key Engineering Benefits	16
5.2 Register File (regfile)	16
5.2.1 Architectural Function & Mechanics	16
5.2.2 Design Rationale: Why We Used It (Problem Solved)	17
5.2.3 Key Engineering Benefits	17
5.3 16-Bit Arithmetic Logic Unit (ALU)	17
5.3.1 Architectural Function & Mechanics	17
5.3.2 Design Rationale: Why We Used It (Problem Solved)	18
5.3.3 Key Engineering Benefits	18
5.4 Integrated Clock Gating Cell (CLK_GATE)	18
5.4.1 Architectural Function & Mechanics	18
5.4.2 Design Rationale: Why Latch-Based ICG Is Mandatory vs. Simple AND Gate	19
5.4.3 Key Engineering Benefits	19
5.5 Clock Divider and Prescaler Subsystem (CLK_DIV & prescale_mux)	20
5.5.1 Architectural Function & Mechanics	20
5.5.2 Design Rationale: Why We Used It (Problem Solved)	20
5.5.3 Key Engineering Benefits	20
5.6 Edge Detector and Pulse Generator (Pulse_Gen)	20
5.6.1 Architectural Function & Mechanics	20

5.6.2	Design Rationale: Why We Used It (Problem Solved)	20
5.6.3	Key Engineering Benefits	20
5.7	UART Transmitter Subsystem (UART_TX)	21
5.7.1	Architectural Function & Mechanics	21
5.7.2	Design Rationale: Why We Used It (Problem Solved)	21
5.7.3	Key Engineering Benefits	21
5.8	UART Receiver Subsystem (UART_RX)	21
5.8.1	Architectural Function & Mechanics	21
5.8.2	Design Rationale: Why 3-Sample Majority Voting Is Essential	22
5.8.3	Key Engineering Benefits	22
6	Architectural Design-Space Exploration and Trade-Off Analysis	22
7	RTL Verification Methodology and Testbench Architecture	23
7.1	Automated Self-Checking Testbench	23
8	Simulation Results and Waveform Analysis	25
9	Synopsys SpyGlass Static Sign-off (Lint and CDC)	27
9.1	SpyGlass Lint Verification	27
9.2	SpyGlass CDC Verification	28
10	Synopsys Design Compiler Logic Synthesis	28
10.1	Synthesis Constraints and Corner Conditions	28
10.2	Timing Closure and Area Results	28
11	Synopsys DFT Compiler (Design-for-Testability)	29
12	Physical Design Implementation: Place and Route and Sign-off	30
12.1	Flow Overview and Setup	30
12.2	Floorplanning and Power Planning	32
12.3	Placement	33
12.4	Clock Tree Synthesis	33
12.5	Routing and Chip Finish	34
12.6	Physical Verification and Sign-off	35
12.7	GDSII Export and Deliverables	36
12.8	Gate-Level Simulation (Post-Route SDF)	37
13	Synopsys Formality Formal Equivalence Verification	37
14	Power Optimization and Dynamic Power Profiling	38
14.1	VCD-Annotated Power Analysis (PrimeTime PX)	38
15	Summary of Implementation Results and Sign-off Metrics	39
16	Conclusion and Future Work	39
16.1	Limitations	40
16.2	Future Work	40
Appendix A — Implementation Flow Scripts		42
Appendix B — Regression Transcript (Excerpt)		42
Appendix C — Reproducibility and Tool Versions		43

List of Figures

1	System Architecture showing clock domain partitioning, CDC synchronization boundaries, and internal bus interconnect.	9
2	ASYNC_FIFO Architecture highlighting Gray-Code Pointer Synchronization. . .	12
3	System Controller Finite State Machine state transition flow.	16
4	UART Receiver Data Path and Error Checking Flow.	21
5	ModelSim Simulation Waveform 1: Clocks & Resets (Gold/Red), Top-Level UART Serial I/O (Cyan/Green), UART RX/TX Subsystem, DATA_SYNC CDC Handshake, and SYS_CTRL Master FSM State Transitions across the complete test run.	25
6	ModelSim Simulation Waveform 2: Master System Controller address/control generation, Register File write/readback transactions, and ALU 16-bit execution with gated clock activation.	26
7	ModelSim Simulation Waveform 3: Asynchronous FIFO dual-clock buffering, write increment (<code>winc</code>), read increment (<code>rinc</code>), and full/empty flag status during burst transfers.	27
8	Complete RTL-to-GDSII implementation and sign-off flow (Cadence Encounter 8.10 with NanoRoute; physical sign-off checks; formal equivalence across the post-synthesis, post-DFT, and post-route netlists).	31
9	SYS_TOP floorplan with the power distribution network — floorplan view captured from the Encounter database (die $240.47 \times 220.47 \mu\text{m}$; power ring and METAL6 straps over the standard-cell rows).	33
10	SYS_TOP final routed layout — physical view captured from the Encounter database (standard-cell rows with clock tree and multi-layer signal routing; die $240.47 \times 220.47 \mu\text{m}$).	35

List of Tables

1	Top-Level Interface Signals of the Subsystem.	8
2	System Clock Domains, Frequencies, and Synchronized Reset Mapping.	9
3	System Command Set and Protocol Definitions.	15
4	ALU Function Opcode Mapping.	18
5	Design-Space Exploration: Clock Domain Crossing (CDC) Strategies.	22
6	Design-Space Exploration: Clock Gating Architectures.	22
7	Design-Space Exploration: Asynchronous Reset Distribution.	23
8	Design-Space Exploration: FIFO Pointer Synchronization Encoding.	23
9	Executed Verification Cases.	24
10	Synthesized Gate-Level Area Breakdown in TSMC 130nm CMOS (global cell area including sub-cells; source: Synthesis/reports/area.rpt).	29
11	Synopsys DFT Compiler Testability Sign-off Metrics.	30
12	Cadence Encounter Physical Implementation Flow Parameters.	32
13	Post-Route Sign-off Results for SYS_TOP	36
14	Exported Implementation Deliverables and File Sizes.	37
15	PrimeTime PX VCD-annotated power summary for the post-route netlist (functional workload).	38
16	Consolidated SoC Architectural & ASIC Implementation Sign-off Metrics.	39
17	Tool versions and technology library used to produce the reported results.	43

Abbreviations

Acronym	Definition
ALU	Arithmetic Logic Unit
ASIC	Application-Specific Integrated Circuit
ATE	Automatic Test Equipment
ATPG	Automatic Test Pattern Generation
CDC	Clock Domain Crossing
CMOS	Complementary Metal-Oxide-Semiconductor
CTS	Clock Tree Synthesis
DC	Synopsys Design Compiler
DFT	Design for Testability
DRC	Design Rule Check
DRV	Design Rule Violation
EDA	Electronic Design Automation
FIFO	First-In, First-Out
FSM	Finite State Machine
GDSII	Graphic Data System II (layout stream format)
GLS	Gate-Level Simulation
ICG	Integrated Clock Gating
LEC	Logical Equivalence Checking
LVS	Layout Versus Schematic
MMMC	Multi-Mode Multi-Corner
MTBF	Mean Time Between Failures
PnR	Place and Route
PVT	Process, Voltage, and Temperature
RTL	Register Transfer Level
SDF	Standard Delay Format
SDFF	Scan (Multiplexed-D) Flip-Flop
SoC	System-on-Chip
SPF	Standard Parasitic Format
STA	Static Timing Analysis
TNS	Total Negative Slack
UART	Universal Asynchronous Receiver/Transmitter
VCD	Value Change Dump (activity trace format)
WNS	Worst Negative Slack

Abstract

This report presents the complete microarchitectural specification, RTL design, directed self-checking functional verification, and complete RTL-to-GDSII ASIC back-end implementation of an industrial-grade, multi-clock domain digital communication System-on-Chip (SoC) subsystem. The design bridges two physically asynchronous clock domains: a high-speed computational domain driven by a reference clock (`REF_CLK` at 100 MHz) and an asynchronous serial interface domain driven by a dedicated baud generator clock (`UART_CLK` at 3.6864 MHz). The hardware architecture comprises an asynchronous multi-frame UART receiver and transmitter with re-configurable oversampling ($8\times$, $16\times$, $32\times$) and parity checking; a double-flop enable-handshake Multi-Bit Data Synchronizer; a 10-state master System Controller FSM; a dual-port 16×8 -bit Register File; an energy-efficient 16-bit Arithmetic Logic Unit (ALU); an 8-word (8×8) dual-clock Asynchronous FIFO utilizing 2-stage Gray-code pointer synchronization; and an Integrated Clock Gating (ICG) cell targeting dynamic power reduction during computational dormancy.

The subsystem was verified in QuestaSim/ModelSim using an automated, directed, self-checking testbench covering reset, register-file write/readback transactions across all parity configurations, and full transmit-frame validation (start, LSB-first data, programmable parity, stop), with all executed checks passing. Static lint and Clock Domain Crossing (CDC) sign-off were achieved using Synopsys SpyGlass under the `GuideWare/latest/block/rtl_handoff` methodology, with zero error-class findings, no unsynchronized clock-domain crossings, and all reported messages reviewed and waived (including the Gray-coded FIFO pointer convergence, confirmed safe by the Gray-encoding check). Logic synthesis was performed in Synopsys Design Compiler targeting the TSMC 130nm CMOS standard-cell library (`scmetro_tsmc_cl013g_rvt`) at the SS / 1.08 V / 125°C corner, achieving zero-slack timing closure at 100 MHz (**Setup Slack = 0.00 ns**, **Hold Slack = +0.43 ns**) across 2,275 standard cells ($27,225.31 \mu\text{m}^2$ of active cell area). Manufacturing testability was integrated via Synopsys DFT Compiler with four balanced scan chains of 95 cells each (380 scan flip-flops total), achieving **99.66% test coverage** across **17,638 faults**. A complete Cadence Encounter place-and-route flow — floorplanning, power planning, placement, clock tree synthesis, and NanoRoute detailed routing — produced a signed-off core layout of $240.47 \times 220.47 \mu\text{m}$ with positive post-route slack (**Setup +4.409 ns**, **Hold +0.015 ns**), zero tool-level DRC, connectivity, and antenna violations, and GDSII stream-out. Formal equivalence was proven with Synopsys Formality across all three sign-off stages — 379 (post-synthesis), 361 (post-DFT) and 361 (post-place-and-route) passing compare points. The exported netlist was additionally re-verified at gate level with fully back-annotated post-route delays (slow-corner SDF) — passing the complete directed regression — and its recorded switching activity drove a PrimeTime PX power analysis of functional-mode operation, reporting a leakage-dominated average power of $\approx 11.7 \mu\text{W}$ at the typical corner (lower bound pending full activity coverage).

Keywords – Multi-Clock SoC, Clock Domain Crossing (CDC), Asynchronous FIFO, Integrated Clock Gating (ICG), TSMC 130nm CMOS, Logic Synthesis, Design for Testability (DFT), Formal Verification, Synopsys Design Compiler, SpyGlass CDC, Place and Route, Gate-Level Simulation, PrimeTime PX, GDSII.

1 Introduction and Project Scope

Modern System-on-Chip (SoC) architectures routinely integrate heterogeneous processing subsystems that operate across multiple asynchronous clock domains. While high-performance computational engines (such as DSPs, vector processors, and arithmetic pipelines) demand high frequencies (100 MHz and beyond) to maximize arithmetic throughput, external peripheral

interfaces (such as UART, SPI, and I2C) are constrained by standard physical baud rates and external crystal oscillators operating at significantly lower frequencies (e.g., 3.6864 MHz). Bridging the boundaries between these asynchronous timing domains represents one of the most critical challenges in digital VLSI design. Asynchronous boundary failures—including flip-flop metastability, multi-bit data incoherency, and reset recovery/removal violations—can cause catastrophic, non-deterministic system failures that completely escape conventional cycle-based RTL simulation.

This final project delivers a complete, synthesizable, industrial-grade System-on-Chip digital communication subsystem implemented from microarchitectural conception to a signed-off, core-level GDSII layout through a complete RTL-to-GDSII back-end flow in **TSMC 130nm CMOS** technology. The core architectural objectives established for this system include:

1. **Robust Clock Domain Crossing (CDC) Safety:** Elimination of metastability and multi-bit data incoherency across asynchronous clock domains using an enable-handshake Multi-Bit Data Synchronizer, dual-domain asynchronous reset synchronizers, and a dual-clock circular FIFO with Gray-coded pointer synchronization.
2. **Aggressive Dynamic Power Reduction:** Architectural integration of an active-low latch-based Integrated Clock Gating (ICG) cell that cuts clock tree switching to the 16-bit ALU whenever arithmetic computation is idle, directly targeting switching power ($P_{\text{dyn}} = \alpha CV^2 f$).
3. **Flexible Serial Telemetry & Dynamic Prescaling:** A fully configurable UART transceiver supporting dynamic software-controlled oversampling ($8\times$, $16\times$, $32\times$), programmable parity generation/checking (Even/Odd/None), framing error detection, and noisy channel filtering via triple-sample majority voting.
4. **Comprehensive Directed Self-Checking Verification:** An automated, directed, self-checking testbench covering reset behavior, register-file write/readback transactions across all parity configurations, and full transmit-frame validation (start, LSB-first data, programmable parity, stop).
5. **Complete ASIC Backend Sign-off Flow:** Static lint and CDC verification via Synopsys SpyGlass, 100 MHz timing closure in Synopsys Design Compiler, scan insertion via Synopsys DFT Compiler, place-and-route closure and physical sign-off culminating in GDSII stream-out, post-route gate-level simulation with SDF back-annotation, activity-annotated power analysis, and formal equivalence verification via Synopsys Formality.

The top-level interface of the subsystem, listed in Table 1, comprises four inputs and three outputs.

Table 1: Top-Level Interface Signals of the Subsystem.

Signal	Direction	Width	Description
REF_CLK	Input	1	High-speed domain reference clock (100 MHz).
UART_CLK	Input	1	Baud generator clock (3.6864 MHz) for the serial domain.
RST	Input	1	Asynchronous active-low reset; synchronized per clock domain.
RX_IN	Input	1	Serial UART receive data input.
TX_OUT	Output	1	Serial UART transmit data output.
parity_error	Output	1	Asserted on receive-frame parity check failure.
framing_error	Output	1	Asserted on invalid stop-bit framing.

The remainder of this report is organized as follows: Sections 2–6 present the system architecture, module-level design, and trade-off analysis; Sections 7–8 cover the verification methodology and functional simulation results; Sections 9–13 describe the implementation and sign-off flow (lint/CDC, synthesis, DFT, place-and-route, post-route gate-level simulation, and formal equivalence); Sections 14–15 report the power analysis — including the VCD-annotated PrimeTime PX results — and the consolidated results summary; and Section 16 concludes the report. Appendices A–C provide implementation scripts, a regression transcript excerpt, and reproducibility details.

2 Multi-Clock System Architecture and Clock Domain Partitioning

2.1 Physical Clock Domain Partitioning

The SoC subsystem is strictly partitioned into two physically asynchronous clock domains to isolate high-frequency arithmetic compute from low-frequency serial telemetry, as illustrated in the system architecture diagram in Figure 1 and summarized in Table 2:

- **Reference Clock Domain (REF_CLK):** The primary high-frequency processing domain operating at 100 MHz. This domain hosts the master System Controller finite state machine (SYS_CTRL), the 16×8 -bit Register File (**regfile**), the 16-bit Arithmetic Logic Unit (ALU), the latch-based clock-gating cell (CLK_GATE), the write-side interface of the Asynchronous FIFO, and the dedicated reference reset synchronizer (RST_SYNC).
- **UART Clock Domain (UART_CLK):** The serial communication domain driven by an external 3.6864 MHz crystal oscillator. This domain hosts the UART Transmitter (UART_TX), the UART Receiver (UART_RX), the TX baud-rate divider, the RX oversampling clock divider, the prescaler multiplexer (**prescale_mux**), the read-side interface of the Asynchronous FIFO, the read-increment pulse generator (**Pulse_Gen**), and the dedicated UART reset synchronizer. The FIFO read side and **Pulse_Gen** are clocked by TX_CLK (the UART_CLK $\div$ 32 baud clock).

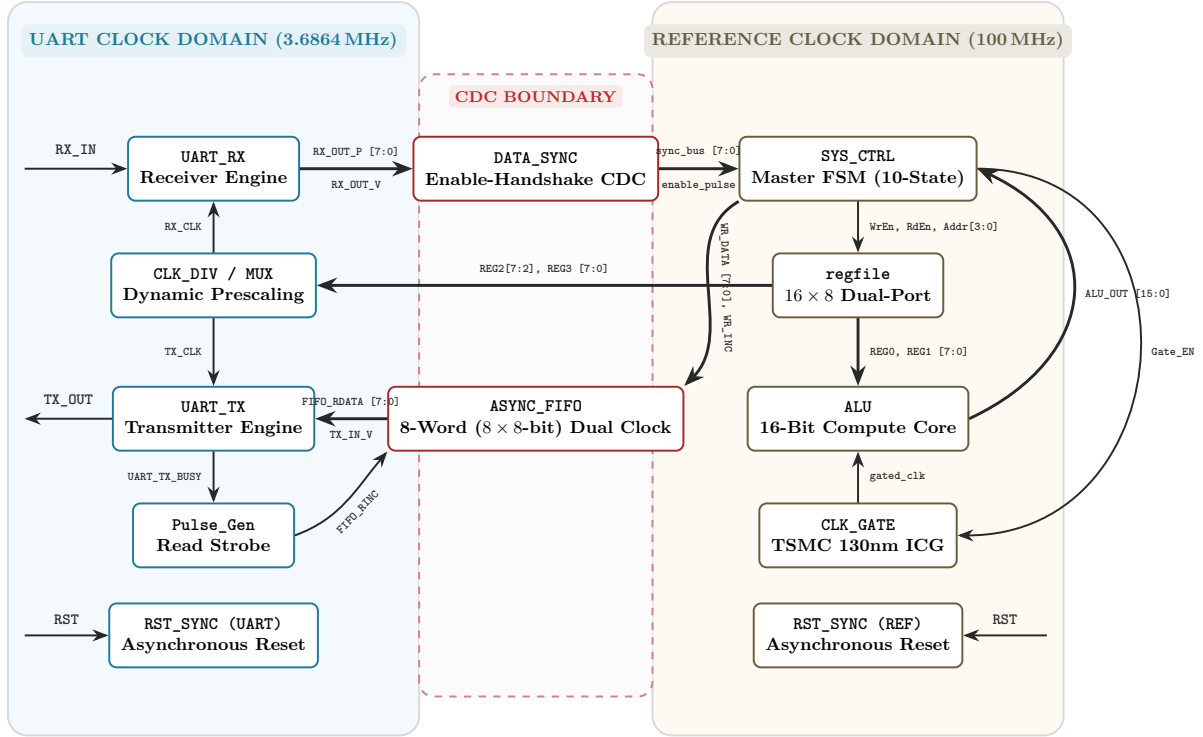


Figure 1: System Architecture showing clock domain partitioning, CDC synchronization boundaries, and internal bus interconnect.

Table 2: System Clock Domains, Frequencies, and Synchronized Reset Mapping.

Domain Name	Source Clock	Frequency	Reset Domain	Hosted Subsystems
Reference Domain	REF_CLK	100 MHz	REF_RST	SYS_CTRL, regfile, ALU, CLK_GATE, FIFO Write Port
UART Serial Domain	UART_CLK	3.6864 MHz	UART_RST	UART_RX, UART_TX, CLK_DIV, Pulse_Gen, FIFO Read Port
Gated Compute Domain	gated_clk	0–100 MHz	REF_RST	16-Bit Arithmetic Core (Internal ALU Switching Logic)

Note: the FIFO read port and Pulse_Gen are clocked by TX_CLK (UART_CLK ÷ 32 baud clock).

3 Comprehensive Clock Domain Crossing (CDC) Synchronization

3.1 Theoretical Foundations: Metastability and Multi-Bit Skew Hazards

When a digital signal transitions asynchronously with respect to the sampling clock of a destination register, setup (t_{su}) or hold (t_h) timing margins may be violated. The destination flip-flop's

internal bistable feedback loop enters a metastable state where its output voltage hovers indeterminately between logical 0 and 1 for an unbounded settling duration. The probability of failure is quantified by the Mean Time Between Failures (MTBF) [2]:

$$\text{MTBF} = \frac{e^{\frac{t_r}{\tau}}}{T_0 \cdot f_{\text{clk}} \cdot f_{\text{data}}} \quad (1)$$

where t_r is the resolving time margin, τ is the flip-flop regenerative time constant, T_0 is an asymptotic thermal factor, f_{clk} is the destination clock frequency, and f_{data} is the asynchronous data transition frequency.

While a single-bit control signal can be protected using a two-stage flip-flop synchronizer, **multi-bit buses cannot be synchronized by placing individual synchronizers on each data line**. Due to non-uniform interconnect routing delays, process variations, and parasitic wire capacitance, individual bits experience unequal skew. If the bus transitions during the destination clock edge, certain bits will resolve in destination cycle N while others resolve in cycle $N + 1$, resulting in catastrophic **bus incoherency** (capturing corrupted, blended data words).

To eliminate metastability and bus incoherency, the system implements three specialized CDC circuits:

3.2 Multi-Bit Data Synchronizer (DATA_SYNC)

3.2.1 Architectural Function & Mechanics

The `DATA_SYNC` module safely transfers the received 8-bit parallel byte (`RX_OUT_P`) and valid flag (`RX_OUT_V`) from the slow 3.6864 MHz `UART_CLK` domain into the high-speed 100 MHz `REF_CLK` domain. It employs an **enable-handshake synchronization topology**:

1. The 8-bit data bus (`RX_OUT_P`) is driven statically by the UART receiver and held constant while the transfer occurs.
2. The single-bit valid strobe (`RX_OUT_V`) is routed into the synchronizer as `bus_enable` and passed through a 2-stage flip-flop synchronizer clocked by `REF_CLK`.
3. An edge-detector flip-flop generates a single-cycle pulse (`enable_pulse`) in the `REF_CLK` domain when the synchronized enable rises:

$$\text{pulse_gen_out} = \text{multi_flops}[1] \wedge \neg \text{enable_flop} \quad (2)$$

4. On `pulse_gen_out`, a multiplexer updates `sync_bus` with `unsync_bus`, latching the fully settled data bus into the destination domain registers.

3.2.2 Design Rationale: Why We Used It (Problem Solved)

Without `DATA_SYNC`, passing an 8-bit received byte directly across the clock boundary would cause multi-bit skew incoherency. An alternative approach would be to instantiate a full dual-clock FIFO at the receiver output. However, because UART frames arrive at a maximum rate of 11,520 bytes/second ($\approx 86.8 \mu\text{s}$ per byte), the destination 100 MHz FSM consumes each byte in tens of nanoseconds. There is zero risk of back-to-back byte collision. Therefore, an enable-handshake synchronizer solves the multi-bit CDC hazard with complete coherency while requiring a fraction of the silicon area of a FIFO.

3.2.3 Key Engineering Benefits

- **100% Bus Coherency:** The multi-bit data lines never transition during the sampling window; only the synchronized control pulse triggers sampling.
- **Silicon Area Efficiency:** Consumes only 3 control flip-flops, 8 data register bits, and minimal gating logic ($\approx 404 \mu\text{m}^2$ synthesized, UO_DATA_SYNC) compared to $\approx 4,103 \mu\text{m}^2$ for the asynchronous FIFO (UO_ASYNC_FIFO).
- **Deterministic Handshake:** Produces an exact 1-clock-cycle `enable_pulse` that directly drives the FSM state transition logic without race conditions.

```
1 module DATA_SYNC #(parameter bus_width = 8, stages = 2) (  
2     input wire clk, rst, bus_enable,  
3     input wire [bus_width - 1 : 0] unsync_bus,  
4     output reg [bus_width - 1 : 0] sync_bus,  
5     output reg enable_pulse  
6 );  
7 reg [0 : stages - 1] multi_flops;  
8 reg enable_flop;  
9 wire pulse_gen_out;  
10 wire [bus_width-1:0] mux_out;  
11  
12 // Multi-stage flip-flop synchronizer on bus_enable control strobe  
13 always @(posedge clk or negedge rst) begin  
14     if (!rst) multi_flops <= 'b0;  
15     else multi_flops <= {bus_enable, multi_flops[0 : stages - 2]};  
16 end  
17  
18 // Edge-detector flop for single-cycle pulse generation  
19 always @(posedge clk or negedge rst) begin  
20     if (!rst) enable_flop <= 1'b0;  
21     else enable_flop <= multi_flops[stages-1];  
22 end  
23  
24 assign pulse_gen_out = multi_flops[stages-1] && !enable_flop;  
25 assign mux_out = pulse_gen_out ? unsync_bus : sync_bus;  
26  
27 // Register coherent data bus on synchronized enable pulse  
28 always @(posedge clk or negedge rst) begin  
29     if (!rst) begin  
30         sync_bus <= 'b0;  
31         enable_pulse <= 1'b0;  
32     end else begin  
33         sync_bus <= mux_out;  
34         enable_pulse <= pulse_gen_out;  
35     end  
36 end  
37 endmodule
```

Listing 1: RTL Implementation of the Multi-Bit Data Synchronizer (DATA_SYNC.v).

3.3 Dual-Clock Asynchronous FIFO (ASYNC_FIFO)

3.3.1 Architectural Function & Mechanics

The ASYNC_FIFO bridges the high-speed REF_CLK domain (write side) and the slow UART_CLK domain (read side). When the 16-bit ALU produces execution results or the Register File performs a memory read, the 8-bit bytes are written into the FIFO. The UART transmitter autonomously reads these bytes and serializes them. The module comprises five sub-blocks (Figure 2):

- **FIFO_MEM_CNTRL:** An 8×8 -bit dual-port static register array with synchronous write on `wclk` and combinational read on `raddr`.

- FIFO_WR: Write address counter and full flag generator operating strictly in `wclk`.
- FIFO_RD: Read address counter and empty flag generator operating strictly in `rclk`.
- DF_SYNC (Write-to-Read): Synchronizes the 4-bit Gray write pointer (`wptr_gray`) into the read clock domain (`rq2_wptr`).
- DF_SYNC (Read-to-Write): Synchronizes the 4-bit Gray read pointer (`rptr_gray`) into the write clock domain (`wq2_rptr`).

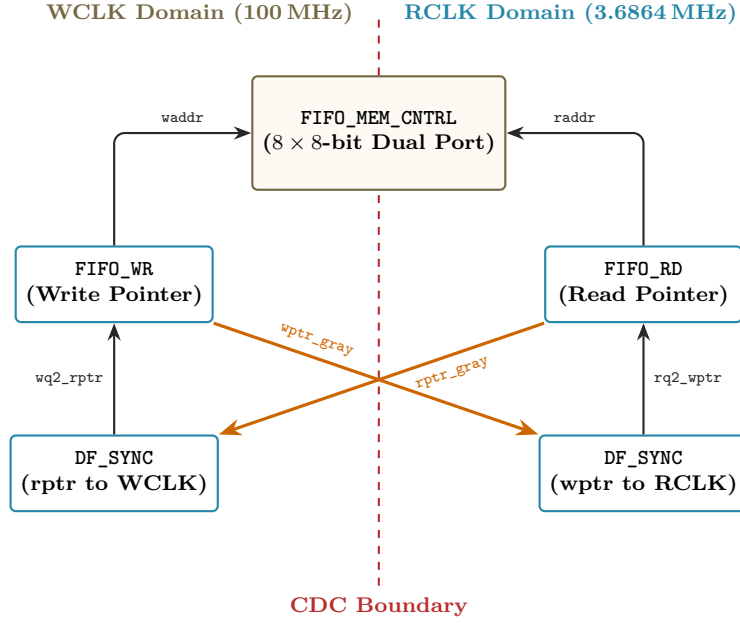


Figure 2: ASYNC_FIFO Architecture highlighting Gray-Code Pointer Synchronization.

3.3.2 Design Rationale: Why We Used It (Problem Solved)

The ALU and System Controller generate 16-bit results as two consecutive bytes in back-to-back 100 MHz clock cycles (20 ns total). Conversely, transmitting a single UART byte at 115,200 baud requires $86.8 \mu\text{s}$ ($\approx 8,680$ reference clock cycles). Without an elastic asynchronous FIFO buffer, the 100 MHz master processor would be forced to enter hard stall states for thousands of cycles waiting for each serial byte to transmit, crippling system computational throughput. The FIFO absorbs burst ALU and Register read responses instantaneously, completely decoupling compute performance from serial line latency.

3.3.3 Why Gray-Coded Pointers Are Mandatory

In standard binary counters, incrementing an address frequently causes multiple bits to toggle simultaneously (for example, transition $0111_2 \rightarrow 1000_2$ flips all 4 bits). When these bits pass across asynchronous clock boundaries, unequal wire delays and flip-flop setup characteristics cause the destination domain to capture transient, erroneous intermediate words (such as 1111_2), falsely triggering full or empty flags and corrupting data pointers.

Gray code guarantees that **exactly one bit changes per address increment**:

$$\text{Gray_Next} = \text{Bin_Next} \oplus (\text{Bin_Next} \gg 1) \quad (3)$$

If the destination clock edge arrives precisely during a pointer bit transition, that bit may resolve to either its previous value or its new value. Because only one bit is changing, the captured Gray

pointer is guaranteed to be either the old pointer value or the new pointer value—**never an arbitrary, illegal intermediate state**. If it resolves to the old pointer, the FIFO simply defers a full or empty release by one cycle, maintaining absolute data integrity.

3.3.4 Cummings' Full and Empty Detection Formulation

The FIFO utilizes $(N + 1)$ -bit pointers for a 2^N -deep memory array ($N = 3$, depth = 8 words) [1]. The $(N + 1)$ -th bit distinguishes between an empty buffer and a full buffer:

- **FIFO Empty Condition** (Evaluated strictly in `rclk` domain):

$$\text{rempty} = (\text{rq2_wptr} == \text{rptr_gray_reg}) \quad (4)$$

- **FIFO Full Condition** (Evaluated strictly in `wclk` domain): The write pointer has wrapped around exactly once and caught up with the synchronized read pointer. In Gray code, this condition is met when the two MSBs are inverted while all lower bits match:

$$\begin{aligned} \text{wfull} = & (\text{wq2_rptr}[3] \neq \text{wgray}[3]) \wedge (\text{wq2_rptr}[2] \neq \text{wgray}[2]) \\ & \wedge (\text{wq2_rptr}[1:0] == \text{wgray}[1:0]) \end{aligned} \quad (5)$$

```

1 module FIFO_WR(
2     input wire      winc, wrst_n, wclk,
3     input wire [3:0] wq2_rptr,
4     output wire [2:0] waddr,
5     output wire [3:0] wptr_gray,
6     output wire      wfull
7 );
8     reg [3:0] wcounter, wptr_gray_reg;
9     reg      wfull_reg;
10
11     wire [3:0] wcounter_next      = (winc && !wfull_reg) ? wcounter + 4'd1 : wcounter;
12     wire [3:0] wcounter_gray_next = wcounter_next ^ (wcounter_next >> 1'b1);
13
14     always @(posedge wclk or negedge wrst_n) begin
15         if (!wrst_n) begin
16             wcounter      <= 4'b0;
17             wptr_gray_reg <= 4'b0;
18             wfull_reg     <= 1'b0;
19         end else begin
20             wcounter      <= wcounter_next;
21             wptr_gray_reg <= wcounter_gray_next;
22             // Cummings Gray-code full condition: top two MSBs inverted, lower bits equal
23             if ((wq2_rptr[3] != wcounter_gray_next[3]) &&
24                 (wq2_rptr[2] != wcounter_gray_next[2]) &&
25                 (wq2_rptr[1:0] == wcounter_gray_next[1:0]))
26                 wfull_reg <= 1'b1;
27             else
28                 wfull_reg <= 1'b0;
29         end
30     end
31     assign wptr_gray = wptr_gray_reg;
32     assign waddr     = wcounter[2:0];
33     assign wfull     = wfull_reg;
34 endmodule

```

Listing 2: Write Pointer and Full Flag Generation (FIFO_WR.v).

3.4 Dual-Domain Asynchronous Reset Synchronizers (RST_SYNC)

3.4.1 Architectural Function & Mechanics

The chip features an active-low global reset input (RST). Because this reset drives flip-flops in two asynchronous clock domains (REF_CLK and UART_CLK), two independent reset synchronizers are instantiated. Each synchronizer consists of a cascade of flip-flops configured with asynchronous reset pins tied to the external RST and data inputs tied to logic 1 (VDD).

3.4.2 Design Rationale: Why We Used It (Problem Solved)

Reset design involves an inherent dilemma:

- **Pure Synchronous Reset:** Cannot reset the chip if the external clock oscillator is dead, unstable, or delayed during initial power-up.
- **Pure Asynchronous Reset:** When released asynchronously, if the deasserting reset edge falls within the setup/hold window of the clock edge, it violates flip-flop recovery time (t_{rec}) or removal time (t_{rem}). This causes random flip-flops to exit reset on clock cycle N while adjacent flip-flops exit on cycle $N + 1$, throwing state machines into illegal, deadlocked states.

The RST_SYNC topology implements **Asynchronous Assertion with Synchronous Deassertion** [9]:

1. When RST drops low, all flip-flops assert reset **instantaneously** via their asynchronous clear pins, without waiting for clock edges.
2. When RST goes high, the logic 1 on the D-input shifts through the multi-stage flop chain, releasing reset **synchronously** aligned to the local clock edge, completely eliminating recovery/removal violations.

```
1 module RST_SYNC #(parameter stages = 3) (  
2     input wire rst, clk,  
3     output wire sync_rst  
4 );  
5     reg [0 : stages - 1] flops;  
6  
7     always @(posedge clk or negedge rst) begin  
8         if (!rst)  
9             flops <= 'b0;                                // Instantaneous asynchronous  
10            assertion  
11        else  
12            flops <= {1'b1, flops[0 : stages - 2]};        // Synchronous deassertion on clock  
13            edge  
14        end  
15        assign sync_rst = flops[stages-1];  
16    endmodule
```

Listing 3: RTL Implementation of the Reset Synchronizer (RST_SYNC.v).

4 Master Communication Protocol and Frame Formats

4.1 UART Serial Frame Specification

Communication between the external host and the SoC occurs over a full-duplex, asynchronous serial link. Each serial frame consists of 10 or 11 bits depending on dynamic parity configuration:

- **Start Bit:** 1 low bit (1'b0) indicating frame transmission start.
- **Data Payload:** 8 data bits transmitted strictly **Least Significant Bit (LSB) first**.
- **Parity Bit (Configurable):** 1 bit enabled via REG2[0] with Even (REG2[1]=0) or Odd (REG2[1]=1) parity.
- **Stop Bit:** 1 high bit (1'b1) verifying framing integrity.

4.2 Baud Rate and Oversampling Calculations

Transmitter baud timing and receiver oversampling are synthesized directly from `UART_CLK` (3.6864 MHz) using programmable integer division ratios:

$$\text{TX Baud Rate} = \frac{f_{\text{UART_CLK}}}{\text{TX Divisor (REG3)}} = \frac{3,686,400 \text{ Hz}}{32} = 115,200 \text{ Baud} \quad (6)$$

For the receiver, an oversampling factor ($M \in \{8, 16, 32\}$) is configured in `REG2[7:2]`. The prescaler multiplexer translates this into an integer divider ratio for `CLK_DIV`:

$$f_{\text{RX_CLK}} = \frac{f_{\text{UART_CLK}}}{\text{Prescale Mux Ratio}} = 115,200 \times M \quad (7)$$

4.3 Master System Command Set

The master controller executes four primary multi-byte command sequences, detailed in Table 3.

Table 3: System Command Set and Protocol Definitions.

Command	Opcode	RX Bytes	TX Bytes	Functional Operation Sequence
RF Write	0xAA	3	0	[Opcode → Address → Data]. Writes 8-bit data into register 0x0–0xF.
RF Read	0xBB	2	1	[Opcode → Address]. Reads addressed register; SoC pushes data to FIFO → <code>UART_TX</code> .
ALU with Op	0xCC	4	2	[Opcode → Op A → Op B → ALU FUN]. Writes <code>REG0</code> , <code>REG1</code> , executes ALU, pushes 16-bit result (MSB then LSB) to FIFO.
ALU without Op	0xDD	2	2	[Opcode → ALU FUN]. Executes ALU on cached <code>REG0</code> , <code>REG1</code> ; pushes 16-bit result (MSB then LSB) to FIFO.

5 Detailed Subsystem Hardware Architecture

5.1 System Controller (SYS_CTRL)

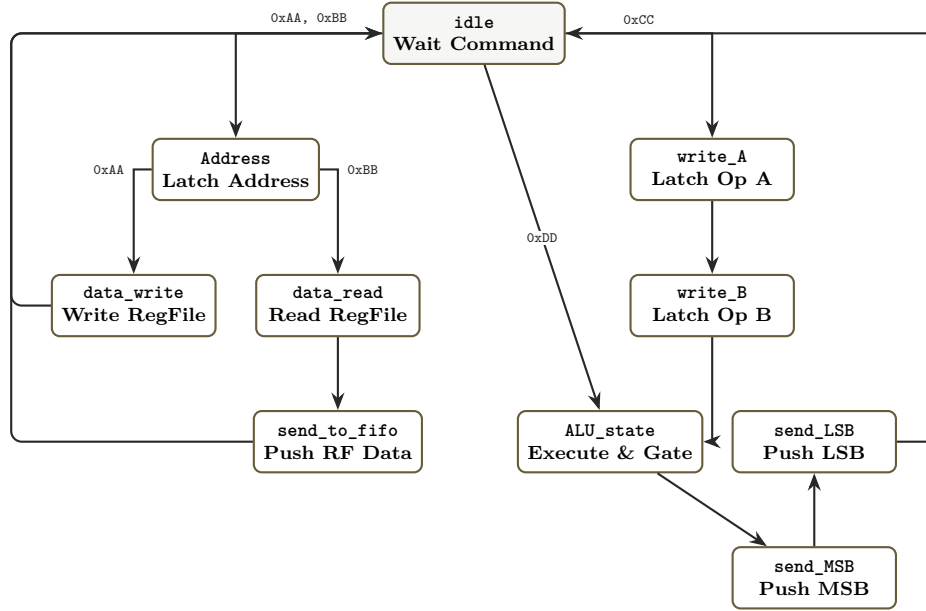


Figure 3: System Controller Finite State Machine state transition flow.

5.1.1 Architectural Function & Mechanics

Operating in the REF_CLK domain, SYS_CTRL acts as the master coordinator finite state machine. It contains 10 states: idle, Address, data_write, data_read, send_to_fifo, write_A, write_B, ALU_state, send_LSB, and send_MSB (Figure 3).

5.1.2 Design Rationale: Why We Used It (Problem Solved)

Serial commands arrive sequentially over thousands of clock cycles. A centralized FSM is mandatory to enforce strict multi-byte protocol framing, route incoming data into the appropriate internal destinations, manage register access strobes (WrEn, RdEn), dynamically assert ALU clock gate enables (Gate_EN), and sequence 16-bit split-byte pushes into the FIFO.

5.1.3 Key Engineering Benefits

- **Deterministic Protocol Sequencing:** Implements unambiguous multi-byte state handling that rejects malformed commands.
- **Atomic Multi-Byte Operations:** Automatically splits 16-bit ALU outputs into ordered MSB-then-LSB byte transfers.
- **Autonomous Power Management:** Directly asserts Gate_EN strictly during active computational states.

5.2 Register File (regfile)

5.2.1 Architectural Function & Mechanics

The Register File provides 16 directly addressable 8-bit registers (128 bits total storage). It features separate write and read ports:

- REG0 (Address 0x0): Dedicated Operand A for the 16-bit ALU.
- REG1 (Address 0x1): Dedicated Operand B for the 16-bit ALU.
- REG2 (Address 0x2): UART Configuration: REG2[0] (Parity Enable), REG2[1] (Parity Type: 0=Even, 1=Odd), REG2[7:2] (Prescaler Division Ratio).
- REG3 (Address 0x3): Transmitter Baud Division Ratio (default value: 32 for 115,200 Baud).
- REG4–REG15 (Addresses 0x4–0xF): General-purpose storage registers for user application data.

5.2.2 Design Rationale: Why We Used It (Problem Solved)

Without a centralized register file, peripheral configuration parameters (baud rate, parity mode, prescaler ratios) would have to be hardwired at synthesis, eliminating operational flexibility. The Register File memory-maps peripheral control, enabling software to dynamically alter communication parameters at runtime while providing dedicated zero-wait-state ports directly into the ALU.

5.2.3 Key Engineering Benefits

- **Zero-Cycle Read Latency:** Provides instantaneous combinational operand delivery to the ALU.
- **Run-Time Peripheral Reconfigurability:** Allows on-the-fly baud rate and parity tuning via standard UART commands.
- **Dedicated Port Isolation:** Prevents ALU data fetches from contending with serial host register accesses.

5.3 16-Bit Arithmetic Logic Unit (ALU)

5.3.1 Architectural Function & Mechanics

The ALU operates on two 8-bit operands (A and B) and yields a full-precision 16-bit output (ALU_OUT) alongside an output valid strobe (out_valid). It supports 16 distinct operations grouped into four categories, summarized in Table 4.

Table 4: ALU Function Opcode Mapping.

Category	ALU_FUN	Operation	Mathematical Description
Arithmetic	4'b0000	Addition	$ALU_OUT = A + B$
	4'b0001	Subtraction	$ALU_OUT = A - B$
	4'b0010	Multiplication	$ALU_OUT = A \times B$ ($8 \times 8 \rightarrow 16$ -bit full precision)
	4'b0011	Division	$ALU_OUT = A/B$ (returns 16'd0 when $B = 0$)
Logical	4'b0100	Bitwise AND	$ALU_OUT = \{8'b0, A \wedge B\}$
	4'b0101	Bitwise OR	$ALU_OUT = \{8'b0, A \vee B\}$
	4'b0110	Bitwise NAND	$ALU_OUT = \{8'b0, \neg(A \wedge B)\}$
	4'b0111	Bitwise NOR	$ALU_OUT = \{8'b0, \neg(A \vee B)\}$
	4'b1000	Bitwise XOR	$ALU_OUT = \{8'b0, A \oplus B\}$
	4'b1001	Bitwise XNOR	$ALU_OUT = \{8'b0, \neg(A \oplus B)\}$
Comparison	4'b1010	Equality	$ALU_OUT = (A == B) ? 16'd1 : 16'd0$
	4'b1011	Greater Than	$ALU_OUT = (A > B) ? 16'd2 : 16'd0$
	4'b1100	Less Than	$ALU_OUT = (A < B) ? 16'd3 : 16'd0$
Shift / Pass	4'b1101	Shift Right	$ALU_OUT = \{8'b0, A >> 1\}$
	4'b1110	Shift Left	$ALU_OUT = \{8'b0, A << 1\}$
	4'b1111	Reserved (default)	Unused encoding — output forced to 16'd0 and out_valid cleared

5.3.2 Design Rationale: Why We Used It (Problem Solved)

The 16-bit ALU serves as the primary hardware accelerator. Dedicated 16-bit output width eliminates numerical truncation and overflow hazards during arithmetic addition and multiplication, returning complete 16-bit mathematical precision to the host computer.

5.3.3 Key Engineering Benefits

- **Single-Cycle Combinational Execution:** Produces full 16-bit arithmetic and logical results in 1 clock cycle.
- **Zero Overflow Hazard:** 16-bit output accommodates the maximum product of two 8-bit numbers ($255 \times 255 = 65,025$).
- **Gated Clocking Interface:** Fully isolated by the clock gating subsystem, drawing near-zero dynamic power when unselected.

5.4 Integrated Clock Gating Cell (CLK_GATE)

5.4.1 Architectural Function & Mechanics

The CLK_GATE module implements latch-based clock gating. In synthesis mode (SYNTHESIS=1), it instantiates the TSMC 130nm library cell TLATNCAX2M. In behavioral simulation mode

(SYNTHESIS=0), it evaluates an active-low transparent latch driving an AND gate:

$$\text{gated_clk} = \text{clk} \wedge \text{latch} \quad (8)$$

5.4.2 Design Rationale: Why Latch-Based ICG Is Mandatory vs. Simple AND Gate

A common misconception in digital design is gating a clock tree using a simple 2-input AND gate between `clk` and `clk_en`. **A simple AND gate produces catastrophic clock glitches:** if `clk_en` transitions from 0 to 1 while `clk` is high, the output generates an immediate, truncated narrow clock pulse. This narrow glitch violates the minimum pulse-width requirement of destination flip-flops, causing unpredictable state corruption.

The active-low latch solves this hazard:

1. When `clk` is LOW, the latch is transparent, allowing `clk_en` to propagate to the latch output.
2. When `clk` transitions HIGH, the latch becomes **opaque**, freezing its state and holding the enable signal rock-solid throughout the entire clock-high phase.
3. Even if `clk_en` changes wildly during clock-high, the latch suppresses the transitions, guaranteeing 100% glitch-free clock gating.

5.4.3 Key Engineering Benefits

- **Massive Dynamic Power Reduction:** Completely halts internal clock switching in the 16-bit multiplier, adders, and output registers during non-ALU cycles, significantly reducing dynamic switching activity during idle periods (power analysis in Section 14).
- **100% Glitch Immunity:** Eliminates spurious clock hazards through active-low latch hold timing.
- **Standard-Cell Synthesis Compatibility:** Directly maps to TSMC 130nm standard-cell ICG primitives recognized by Synopsys Power Compiler.

```
1 module CLK_GATE #(parameter SYNTHESIS = 0) (  
2     input wire clk, clk_en,  
3     output wire gated_clk  
4 );  
5 generate  
6     if (SYNTHESIS) begin : GEN_SYNTHESIS  
7         // Instantiation of TSMC 130nm standard-cell Integrated Clock Gating primitive  
8         TLATNCAX2M U0 (.CK(clk), .E(clk_en), .ECK(gated_clk));  
9     end else begin : GEN_SIMULATION  
10        reg latch;  
11        // Active-low transparent latch prevents clock glitches  
12        always @(clk or clk_en) begin  
13            if (!clk) latch <= clk_en;  
14        end  
15        assign gated_clk = clk && latch;  
16    end  
17 endgenerate  
18 endmodule
```

Listing 4: RTL Implementation of the Integrated Clock Gating Cell (CLK_GATE.v).

5.5 Clock Divider and Prescaler Subsystem (CLK_DIV & prescale_mux)

5.5.1 Architectural Function & Mechanics

The subsystem contains two instances of CLK_DIV:

- U0_TX_CLK_DIV: Divides UART_CLK by REG3 (default: 32) to generate TX_CLK (115,200 Hz).
- U1_RX_CLK_DIV: Divides UART_CLK by rx_div_ratio (generated by prescale_mux) to generate RX_CLK for oversampling.

The prescale_mux decodes REG2[7:2] to determine the RX clock division factor:

- Prescale = 32 $\implies$ div_ratio = 1 ($f_{RX_CLK} = 3.6864 \text{ MHz} = 32 \times 115,200$).
- Prescale = 16 $\implies$ div_ratio = 2 ($f_{RX_CLK} = 1.8432 \text{ MHz} = 16 \times 115,200$).
- Prescale = 8 $\implies$ div_ratio = 4 ($f_{RX_CLK} = 921.6 \text{ kHz} = 8 \times 115,200$).

5.5.2 Design Rationale: Why We Used It (Problem Solved)

Without on-chip digital dividers, the system would require separate external crystal oscillators for transmission, reception, and each prescale setting, substantially driving up PCB bill-of-materials (BOM) cost and component count. Integrating programmable clock dividers allows a single external 3.6864 MHz oscillator to synthesize all required communication clocks dynamically.

5.5.3 Key Engineering Benefits

- **BOM Cost & Board Space Optimization:** Eliminates external clock generator ICs and auxiliary crystals.
- **50% Duty Cycle Generation:** Internal edge counters achieve exact 50% duty cycle across even division ratios.
- **Dynamic Glitch-Free Reconfiguration:** Detects division ratio changes at runtime and resets counters smoothly without generating spurious clock pulses.

5.6 Edge Detector and Pulse Generator (Pulse_Gen)

5.6.1 Architectural Function & Mechanics

The Pulse_Gen module captures the UART_TX_BUSY signal in the TX_CLK domain. An edge-detect flop asserts FIFO_RINC for **exactly one clock cycle** when UART_TX_BUSY rises, triggering the Asynchronous FIFO to advance its read pointer.

5.6.2 Design Rationale: Why We Used It (Problem Solved)

When the UART transmitter begins serializing an 8-bit frame, its busy flag remains asserted high across the entire 10- or 11-bit transmission window (thousands of clock cycles). If this level signal were routed directly to the FIFO read increment pin, the FIFO would continuously pop words on every cycle, causing immediate buffer underflow and destroying all stored data. The pulse generator converts the sustained level into a clean single-cycle strobe.

5.6.3 Key Engineering Benefits

- **FIFO Underflow Protection:** Ensures exactly one word is read per transmitted UART frame.
- **Interface Standardization:** Robustly converts multi-cycle asynchronous status flags into single-cycle RTL control strobes.

5.7 UART Transmitter Subsystem (UART_TX)

5.7.1 Architectural Function & Mechanics

The transmitter receives 8-bit parallel data from the FIFO when TX_IN_V is asserted. Controlled by FSM_TX, it serializes the data via `Serializer`, appends the start bit (1'b0), computes and appends the parity bit via `Parity_Calc`, and appends the stop bit (1'b1), driving TX_OUT.

5.7.2 Design Rationale: Why We Used It (Problem Solved)

Standard RS-232/UART serial communication provides an industry-standard interface to host PCs, microcontrollers, and debugging terminals without requiring proprietary transceiver hardware.

5.7.3 Key Engineering Benefits

- **Autonomous Frame Sequencing:** Manages bit-level serialization independently of the CPU.
- **Dynamic Parity Computation:** Generates even, odd, or no parity bits on-the-fly based on REG2.
- **Hardware Flow Control:** Asserts UART_TX_BUSY to throttle incoming reads from the FIFO.

5.8 UART Receiver Subsystem (UART_RX)

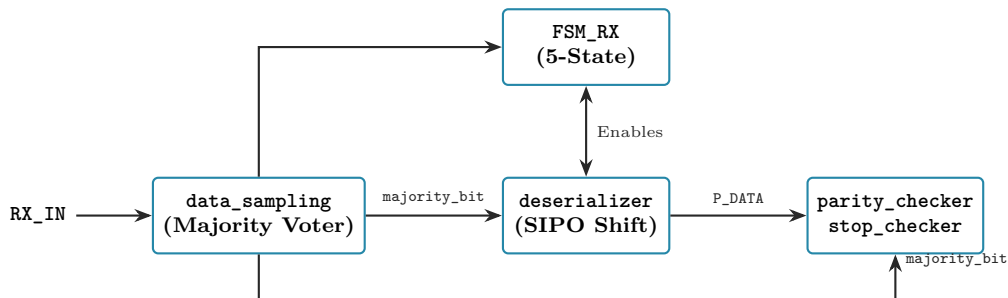


Figure 4: UART Receiver Data Path and Error Checking Flow.

5.8.1 Architectural Function & Mechanics

The receiver captures incoming serial data on RX_IN at 8×, 16×, or 32× oversampling (Figure 4). It comprises seven modular sub-blocks:

- **data_sampling:** Samples the input stream at the center of the bit duration ($N/2 - 1$, $N/2$, $N/2 + 1$) and applies a **three-sample majority voting filter**:

$$\text{majority_bit} = (S_1 \wedge S_2) \vee (S_1 \wedge S_3) \vee (S_2 \wedge S_3) \quad (9)$$

- **strt_checker:** Evaluates the filtered start bit to reject false noise spikes.
- **deserializer:** Shifts 8 serial bits into a parallel byte register.
- **parity_checker:** Checks parity checksum compliance and asserts `parity_error`.
- **stop_checker:** Verifies that the stop bit is logic 1 and asserts `framing_error`.
- **edge_bit_counter:** Tracks intra-bit oversampling edges and inter-bit frame progress.
- **FSM_RX:** 5-state receiver controller (`idle`, `start`, `data`, `parity`, `stop`).

5.8.2 Design Rationale: Why 3-Sample Majority Voting Is Essential

In real-world physical transmission environments, serial interconnects are subjected to electromagnetic interference (EMI), ground bounce, and clock frequency drift. A simple flip-flop sampler that takes a single snapshot at the clock edge is highly susceptible to corrupting entire data frames if a transient noise spike coincides with the sampling instant. The 3-sample majority voter samples the middle of the bit eye three consecutive times and takes the majority value, completely eliminating single-cycle noise glitches.

5.8.3 Key Engineering Benefits

- **High Noise & Spike Immunity:** Filters out line glitches shorter than half a bit period.
- **Real-Time Error Reporting:** Immediately alerts the system to transmission corruption via `parity_error` and `framing_error`.
- **Adaptive Oversampling:** Allows software to trade off receiver power consumption against noise margin ($8\times$ for low power, $32\times$ for maximum noise rejection).

6 Architectural Design-Space Exploration and Trade-Off Analysis

To substantiate the microarchitectural choices made in this SoC, Table 5 through Table 8 provide rigorous comparative trade-off analyses against alternative design options.

Table 5: Design-Space Exploration: Clock Domain Crossing (CDC) Strategies.

CDC Technique	Latency	Bus Safety	Throughput	Implementation Decision & Rationale
Direct Flip-Flop Array	2 cycles	FAIL	High	Rejected: Multi-bit skew causes catastrophic data incoherency.
Asynchronous FIFO	3–4 cycles	PASS	Streaming	Selected for Compute-to-UART Path: Absorbs 100 MHz burst data.
Enable-Handshake Sync	2–3 cycles	PASS	Medium	Selected for UART-to-FSM Path: Coherent multi-bit transfer at minimal area.

Table 6: Design-Space Exploration: Clock Gating Architectures.

Gating Topology	Power Savings	Glitch Risk	Area & Overhead	Implementation Decision & Rationale
Software Multiplexing	None	None	Moderate	Rejected: Clock tree continues toggling; zero dynamic power saved.
Combinational AND Gate	High	SEVERE	Minimal	Rejected: Generates fatal clock glitches on enable transitions.
Latch-Based ICG Cell	Up to 80%	ZERO	Low	Selected: Eliminates glitches via active-low latch; maps to TSMC 130nm cell.

Table 7: Design-Space Exploration: Asynchronous Reset Distribution.

Reset Strategy	Power-On Safety	Recovery Violations	Skew Immunity	Implementation Decision & Rationale
Pure Synchronous	Low	None	Moderate	Rejected: Cannot initialize state if clock is dead at startup.
Pure Asynchronous	High	HIGH	Vulnerable	Rejected: Deassertion violates recovery time, desynchronizing FSMs.
Async Assert / Sync Deassert	High	ZERO	Complete	Selected: Provides immediate reset with glitch-free synchronous release.

Table 8: Design-Space Exploration: FIFO Pointer Synchronization Encoding.

Pointer Encoding	Toggles / Increment	CDC Viability	Area Overhead	Implementation Decision & Rationale
Standard Binary	Up to N bits	UNUSABLE	None	Rejected: Multi-bit transitions cause false full/empty triggers.
One-Hot Encoding	2 bits	POOR	Exponential	Rejected: Area scales exponentially with depth (2^N flops).
Gray Code	Strictly 1 bit	OPTIMAL	Low (XOR)	Selected: Single-bit change guarantees pointer coherency across CDC.

7 RTL Verification Methodology and Testbench Architecture

7.1 Automated Self-Checking Testbench

Functional verification was executed using an automated, self-checking Verilog testbench [3] (SYSTEM_TOP_tb.v) developed for QuestaSim/ModelSim. The testbench features task-based serial stimulus drivers, expected-value checkers, and centralized error logging.

The test sequence executes the directed verification cases listed in Table 9. The protocol cases exercise the complete system path: host command bytes enter through RX_IN and are decoded by the UART receiver and the master System Controller; the requested register content is then returned through the transmit path (including the asynchronous FIFO buffer) and validated on TX_OUT. The testbench is organized around reusable Verilog `task` constructs that emulate the host-side physical interface:

- **send_bit & send_byte:** Drives the RX_IN pin serially, calculating and injecting start, payload, programmable parity, and stop bits correctly synchronized to the RX_CLK generated dynamically based on REG2.
- **write_reg & read_reg_and_check:** Automates multi-byte register transactions using the host command opcodes 0xAA (register write) and 0xBB (register readback), including address transmission and readback validation.
- **check_tx_frame:** Implements robust timeout monitors (20,000 cycles) and sequentially validates the start bit, the eight payload bits (LSB-first), the expected dynamic parity bit when enabled, the stop bit, and the return of TX_OUT to the idle level, without human visual inspection.
- **check_value:** Performs immediate equivalence checking against golden expected bytes, logging failures dynamically.

The archived regression run terminated with the success banner **ALL TESTS PASSED**; the transcript summary reports **Errors: 0, Warnings: 0**. The complete regression transcript

Table 9: Executed Verification Cases.

Test ID	Verification Target	Status	Observed Verification Behavior
TC_01	Reset & Default Check	PASS	Reset released after 1,000 ns; system settles for 100 REF_CLK cycles before the first command.
TC_02	Register File Write/Readback: Parity OFF (0x55)	PASS	0x55 written to REG4 (parity off; REG2=0x80); direct register check and 0xBB readback frame both matched.
TC_03	Second Data Pattern (0xA5)	PASS	0xA5 written to REG5; register check and readback frame both matched.
TC_04	Even Parity (0x3C)	PASS	Even parity enabled (REG2=0x81); 0x3C written to REG6; readback frame carried the correct even parity bit.
TC_05	Odd Parity (0x96)	PASS	Odd parity enabled (REG2=0x83); 0x96 written to REG7; readback frame carried the correct odd parity bit.
TC_06	TX Frame-Format Validation	PASS	Start bit, eight data bits (LSB-first), dynamic parity when enabled, stop bit, and idle return validated on every readback frame (20,000-cycle monitors).

8 Simulation Results and Waveform Analysis

Entity: SYSTEM_TOP_tb Architecture: Date: Thu Oct 01 23:00:42 +0300 2026 Row: 1 Page: 1

25

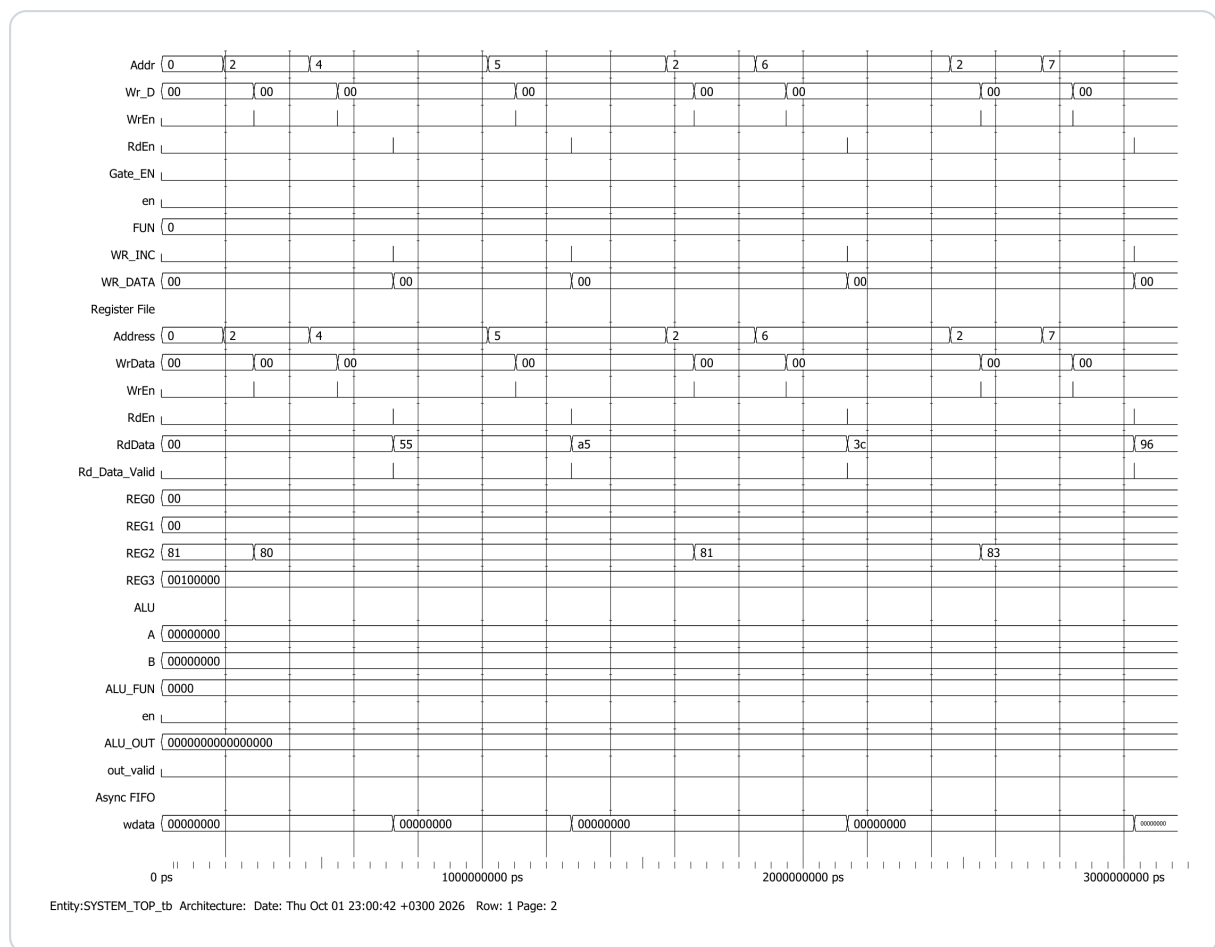


Figure 6: ModelSim Simulation Waveform 2: Master System Controller address/control generation, Register File write/readback transactions, and ALU 16-bit execution with gated clock activation.

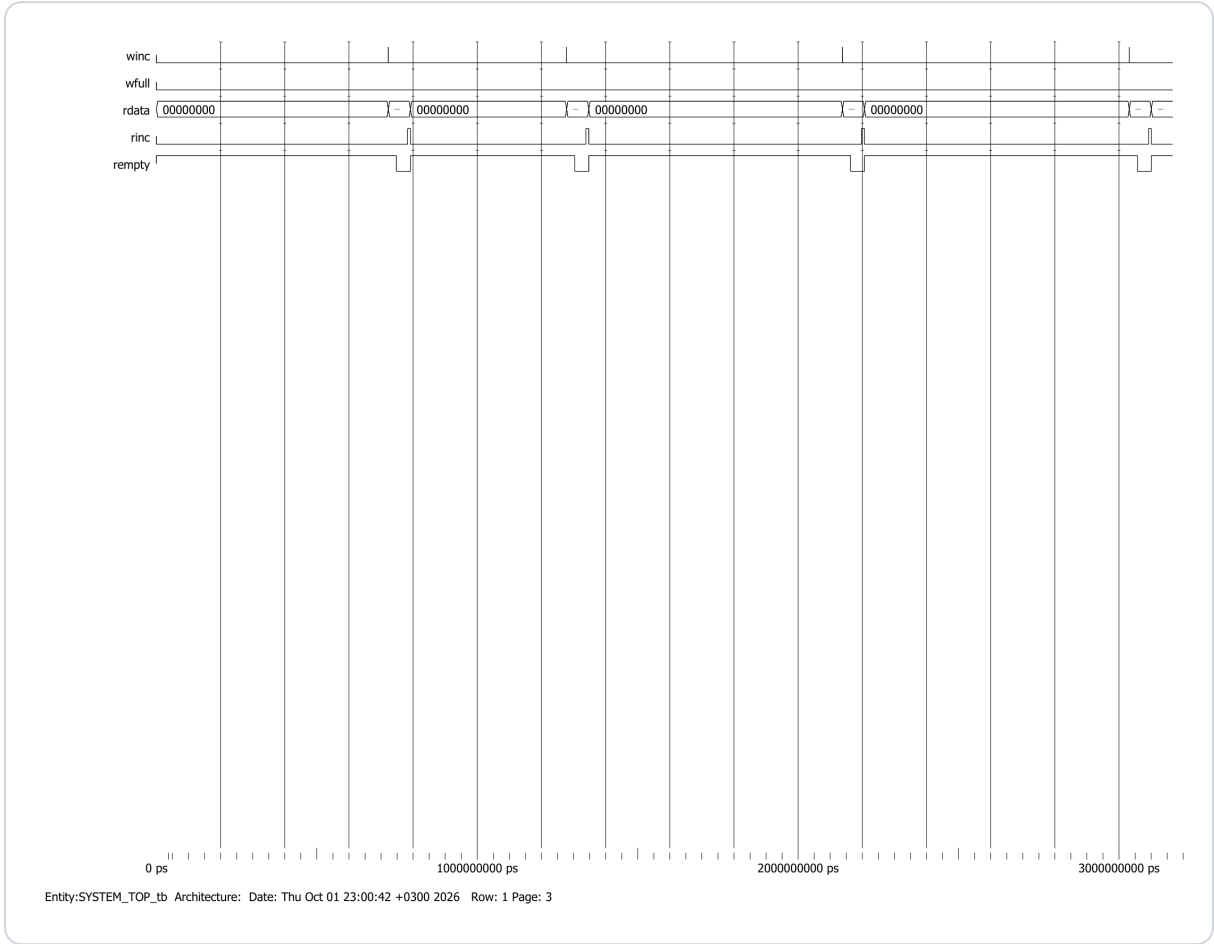


Figure 7: ModelSim Simulation Waveform 3: Asynchronous FIFO dual-clock buffering, write increment (*winc*), read increment (*rinc*), and full/empty flag status during burst transfers.

9 Synopsys SpyGlass Static Sign-off (Lint and CDC)

Prior to logic synthesis, the RTL codebase was verified using Synopsys SpyGlass [7] under the GuideWare/latest/block/rtl_handoff ruleset to ensure lint compliance and structural/functional CDC robustness.

9.1 SpyGlass Lint Verification

The lint run completed with **0 errors and 6 STARC05 warnings** (all waived after review): 2× STARC05-1.4.3.4 — divider output clock used as a non-clock signal (*Clk_Div.v*); 4× STARC05-2.11.3.1 — combined combinational/sequential FSM coding style (*FIFO_RD.v*, *FSM_RX.v*, *Serializer.v*, *deserializer.v*). All were reviewed and waived, with no RTL changes required.

- **No Unintended Latches:** Verified that all combinational always blocks provide complete case/if-else assignments, preventing inferred transparent latches.
- **Full Bus Width Compliance:** Verified that no implicit wire truncations, bit-width mismatches, or multi-driven nets exist.
- **FSM Reachability:** FSM encoding reviewed with default recovery to idle; no unreachable-state findings were reported by the lint ruleset.

9.2 SpyGlass CDC Verification

SpyGlass CDC validated the multi-clock domain crossings across both domains:

- **Synchronization Topology Compliance:** Verified every asynchronous crossing is bound to an approved synchronization structure (two-flop metastability synchronizers, enable-handshake, Gray-coded FIFO pointers, quasi-static configuration paths); 0 unsynchronized crossings.
- **Gray-Code Pointer Verification:** Proved that the 4-bit write and read pointers in `ASYNC_FIFO` strictly change by a Hamming distance of 1 per clock increment.
- **Convergence and Reconvergence Analysis:** One convergence point was identified — the four synchronized Gray write-pointer bits converging on the read counter (`FIFO_RD.rcounter[0]`); the Gray-encoding check PASSED, confirming a hazard-free converged path (structural-run warning waived after review).
- **Data-Enable Sequencing Advisories:** three transmit-path checks (`Ac_datahold01a`; in `Parity_Calc` and `Serializer`) were reported as partially proved with zero failing properties and were waived at sign-off.

10 Synopsys Design Compiler Logic Synthesis

Logic synthesis was conducted in **Synopsys Design Compiler** [4] targeting the **TSMC 130nm CMOS** standard-cell library (`scmetro_tsmc_c1013g_rvt`).

10.1 Synthesis Constraints and Corner Conditions

The design was synthesized at the worst-case operational PVT corner condition:

- **Process Corner:** Slow-Slow (SS) / 1.08 V / 125°C.
- **Reference Clock Constraint:** $f_{\text{REF_CLK}} = 100.0 \text{ MHz}$ ($T_{\text{clk}} = 10.0 \text{ ns}$).
- **UART Clock Constraint:** $f_{\text{UART_CLK}} = 3.6864 \text{ MHz}$ ($T_{\text{clk}} = 271.27 \text{ ns}$).
- **Clock Uncertainty:** Setup uncertainty = 0.2 ns, Hold uncertainty = 0.1 ns.

10.2 Timing Closure and Area Results

The synthesized gate-level netlist achieved complete zero-slack timing closure across all paths:

- **Worst Negative Slack (Setup):** 0.00 ns (MET across all 2,275 cells).
- **Worst Hold Slack:** +0.43 ns (MET).
- **Critical Path:** Register File $\rightarrow$ ALU divider datapath $\rightarrow$ `ALU_OUT[0]` register (9.86 ns data arrival).

A hierarchical breakdown of the synthesized standard-cell area is provided in Table 10.

Table 10: Synthesized Gate-Level Area Breakdown in TSMC 130nm CMOS (global cell area including sub-cells; source: `Synthesis/reports/area.rpt`).

Subsystem	Area (μm^2)	% of Core
ALU (16-bit Engine)	8,864.08	32.6%
RegFile (16×8 -bit)	6,045.88	22.2%
ASYNC_FIFO (Dual-Clock)	4,103.15	15.1%
UART Subsystem (TX + RX)	3,730.14	13.7%
U0_TX_CLK_DIV	1,269.66	4.7%
U1_RX_CLK_DIV	1,269.66	4.7%
SYS_CTRL (Master FSM)	1,203.76	4.4%
Syncs, Pulse_Gen, Mux, CLK_GATE, top glue	738.97	2.7%
Total Core	27,225.31	100.0%

11 Synopsys DFT Compiler (Design-for-Testability)

Manufacturing testability was integrated via **Synopsys DFT Compiler** [5] using standard full-scan insertion methodology:

- **Scan Cell Replacement:** All 380 scan flip-flops were replaced with multiplexed-D scan flip-flops (SDFF) from the TSMC 130nm library; the design’s 380 scan cells are grouped into four fully balanced chains of 95 cells each.
- **Scan Chain Architecture:** Four balanced scan chains, 95 cells each ($\text{SI}[3:0] \rightarrow \text{S0}[3:0]$), were inserted to maximize ATPG pattern shift parallelism and reduce automatic test equipment (ATE) test time.
- **Clock Domain Isolation:** Scan chains were partitioned such that flip-flops belonging to REF_CLK and UART_CLK are isolated during shift mode, preventing scan-shift clock skew race hazards.

The resulting testability sign-off metrics are summarized in Table 11.

Table 11: Synopsys DFT Compiler Testability Sign-off Metrics.

Testability Parameter	Achieved Value	Sign-off Verification Description
Total Scan Flip-Flops	380 Flops	4 balanced chains $\times$ 95 cells.
Scan Chain Lengths	95 / 95 / 95 / 95	Fully balanced (logged by DFT Compiler).
Test Coverage	99.66%	Estimate over 17,638 collapsed faults.
Fault Classes	17,305 DT / 1 PT / 273 UD / 48 AU / 11 ND	Detected / possibly detected / undetectable / ATPG-untestable / not detected.

12 Physical Design Implementation: Place and Route and Sign-off

12.1 Flow Overview and Setup

The physical implementation of `SYS_TOP` was carried out in Cadence First Encounter v08.10-p004 with the embedded NanoRoute v08.10-p008 router [8], starting from the post-DFT gate-level netlist (`DFT/netlists/SYS_TOP.v`) and its timing constraints (`SYS_TOP.sdc`). A multi-mode multi-corner (MMMC) configuration drove all analysis: three constraint modes (functional, scan, and capture) were combined with the minimum and maximum cell libraries (`ff_1p32v_m40c` / `ss_1p08v_125c`; a typical library `tt_1p2v_25c` was also loaded), producing six analysis views (setup and hold views for each mode) over a single extracted RC corner.

The complete session was executed on October 4, 2026 and ran for approximately 19 minutes (23:37–23:55), carrying the design from floorplanning through routing, sign-off, and GDSII export with no unresolved violations. The executed command sequence is archived with the flow scripts (Appendix A). Figure 8 shows the extended RTL-to-GDSII flow, and Table 12 summarizes the place-and-route setup.

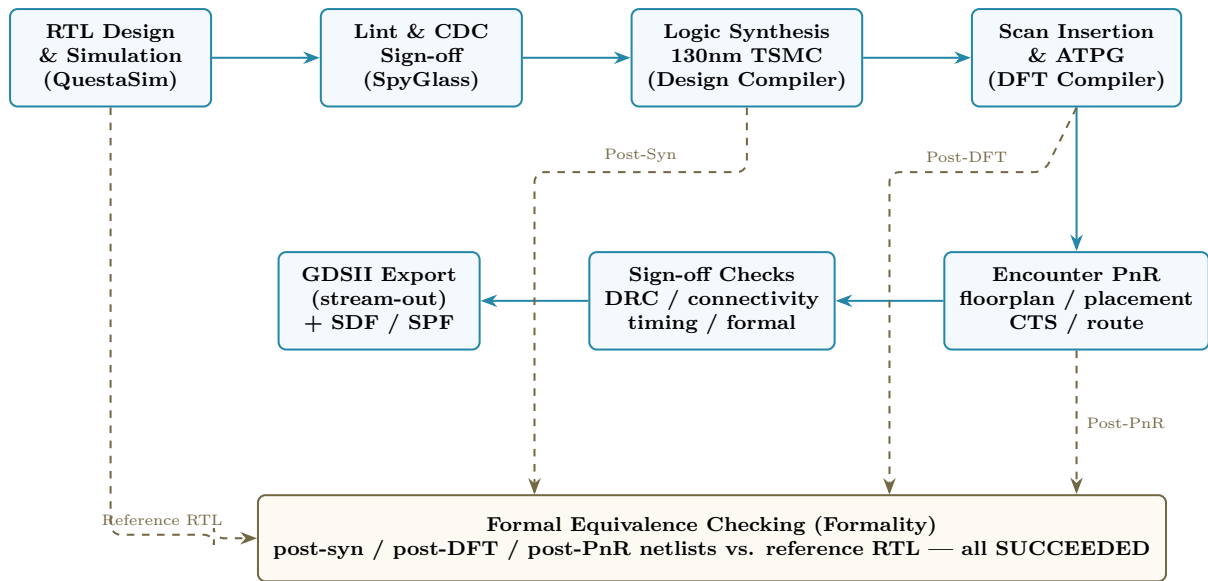


Figure 8: Complete RTL-to-GDSII implementation and sign-off flow (Cadence Encounter 8.10 with NanoRoute; physical sign-off checks; formal equivalence across the post-synthesis, post-DFT, and post-route netlists).

Table 12: Cadence Encounter Physical Implementation Flow Parameters.

Parameter	Value	Source
Tool / version	Cadence First Encounter v08.10-p004 with embedded NanoRoute v08.10-p008	encounter.log
Technology	0.13 μm CMOS, 7-metal FSG stack (tsmc13fsg_71m_tech.lef); Metro c1013g RVT cells	des_import.tcl
Timing corners	min ff_1p32v_m40c, max ss_1p08v_125c, typical tt_1p2v_25c; single extracted RC corner	MMMC.tcl
Analysis views	3 constraint modes (functional / scan / capture) $\times$ 2 corners = 6 views (setup 1–3, hold 1–3)	MMMC.tcl
Die and core	die $240.47 \times 220.47 \mu\text{m}$ with $6 \mu\text{m}$ margins; core box $228.32 \times 208.32 \mu\text{m}$	SYS_TOP.fp
I/O pins	19 ports (11 single-bit + SI[3:0] + SO[3:0])	encounter.log
Row architecture	site TSM130NMMETROSITE, 556 sites per row, $2.87 \mu\text{m}$ row pitch	SYS_TOP.fp
PG ring	width $2 \mu\text{m}$, spacing $0.5 \mu\text{m}$; METAL5 top/bottom + METAL6 left/right; stacked vias METAL1 to METAL7	encounter.log
PG stripes	METAL6, width $1 \mu\text{m}$, pitch $60 \mu\text{m}$, spacing $0.5 \mu\text{m}$	encounter.log
Placement	placeDesign -inPlaceOpt -prePlaceOpt; density 56.8% ($26,751 \mu\text{m}^2$ cell area)	encounter.log
CTS	clockDesign -specFile Clock.ctstch -fixedInstBeforeCTS; skew target 200 ps, transition target 50 ps	Clock.ctstch
Routing	routeDesign -globalDetail -viaOpt -wireOpt; 6 metal layers used (M1–M6)	routing.tcl
Fill	seven FILL sizes (FILL1M–FILL64M), 3,145 filler cells, FIXED mode	encounter.log
Reference clocks	REF_CLK 10 ns (100 MHz); UART_CLK 271.267 ns; scan clock dft_clk 1000 ns	SYS_TOP.sdc
Constraints	setup uncertainty 0.2 ns / hold 0.1 ns; clock transition 0.05 ns; scan I/O delay 200 ns; test_mode = 1, SE = 0 case analysis	SYS_TOP.sdc

12.2 Floorplanning and Power Planning

The die was sized with `floorPlan -d 240.47 220.47 6.0 6.0 6.0 6.0`, producing a $240.47 \times 220.47 \mu\text{m}$ die with nominal $6 \mu\text{m}$ margins on all sides and a core box of $228.32 \times 208.32 \mu\text{m}$. Standard-cell rows use the TSM130NMMETROSITE site with 556 sites per row at a $2.87 \mu\text{m}$ row pitch.

The power distribution network consists of a supply ring $2 \mu\text{m}$ wide (METAL5 along the top and bottom edges, METAL6 along the left and right edges, stitched with stacked vias from METAL1 to METAL7) and METAL6 stripes $1 \mu\text{m}$ wide at a $60 \mu\text{m}$ pitch. Special routing (`sroute`) then connected the supply network, routing 146 core power ports and 73 follow-pin connections; only the warnings expected of a pre-placement power route were reported, and supply integrity was later confirmed clean by the post-route connectivity check discussed below. Figure 9 shows the resulting floorplan with the completed power distribution network.

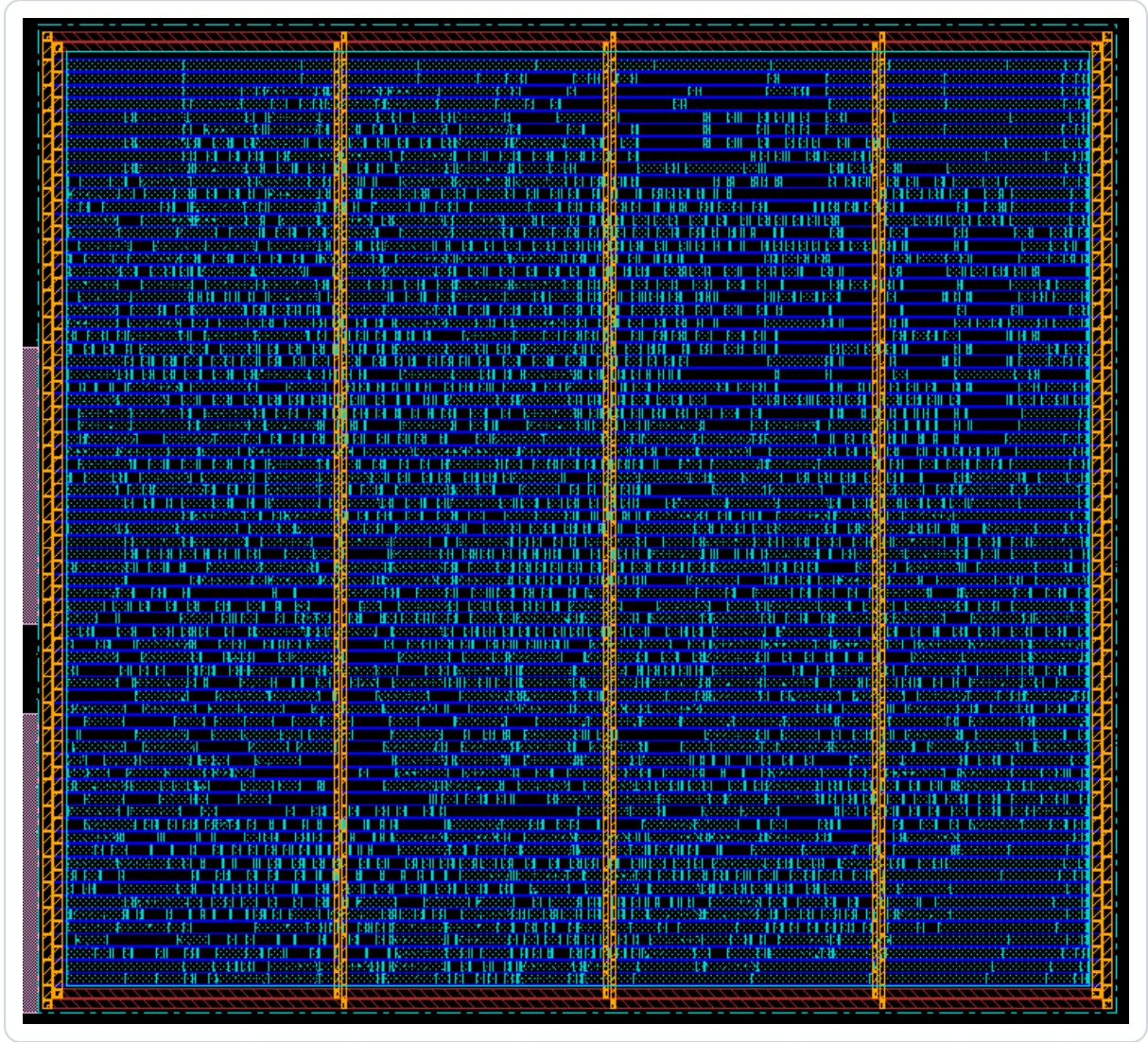


Figure 9: SYS_TOP floorplan with the power distribution network — floorplan view captured from the Encounter database (die $240.47 \times 220.47 \mu\text{m}$; power ring and METAL6 straps over the standard-cell rows).

12.3 Placement

Standard cells were placed with timing-driven optimization (`placeDesign -inPlaceOpt -prePlaceOpt`). The placed design reached a density of 56.8% ($26,751 \mu\text{m}^2$ of standard-cell area) with zero unplaced instances; detailed placement refined 885 instances. Seven tie-off cells ($3 \times \text{TIELOM}$, $4 \times \text{TIEHIM}$) were inserted to tie unused inputs and connected to the power and ground network. The pre-CTS timing check reported a setup worst negative slack (WNS) of +4.315 ns with zero violating paths across all six analysis views.

12.4 Clock Tree Synthesis

Clock tree synthesis (CTS) was specified with a maximum skew target of 200 ps and a sink transition target of 50 ps, and was built with `clockDesign -specFile Clock.ctstch -fixedInstBeforeCTS` for the three clock trees: the scan clock `dft_clk` (reported as `scan_clk`), `UART_CLK`, and `REF_CLK`. Because `-fixedInstBeforeCTS` was used, the clock cells were excluded from subsequent placement changes. The final routed clock trees achieved:

- scan clock: 64 buffers, 376 sinks, 20 levels, worst skew 159 ps (hold 94.3 ps) against the

200 ps requirement;

- UART_CLK: 29 buffers, 110 sinks, 17 levels, 124.7 ps (hold 52.4 ps);
- REF_CLK: 19 buffers, 249 sinks, 9 levels, 13.5 ps (hold 11.6 ps).

The initial post-CTS hold check exposed a worst hold slack of -0.724 ns across 328 violating paths (total negative slack -88.533 ns), which was fixed with `optDesign -postCTS -hold`: 19 cells were added ($14 \times$ DLY4X1M, $1 \times$ DLY2X1M, $2 \times$ DLY1X1M, $2 \times$ BUFX2M) to restore positive hold slack ($+0.014$ ns) without disturbing the setup result.

12.5 Routing and Chip Finish

Routing was executed with NanoRoute using global and detailed routing together with via and wire optimization (`routeDesign -globalDetail -viaOpt -wireOpt`); 84 clock nets (3.62%) were given an additional preferred spacing class to protect the clock network. The final routed design contains $70,072 \mu\text{m}$ of wire length, dominated by METAL2 (29.68 mm) and METAL3 (26.31 mm), and 16,708 vias of which 32.6% are multi-cut; the route completed with 0 DRC violations and 0 route failures after optimization resolved the initial markers.

Chip finish added 3,145 filler cells across seven FILL sizes in FIXED mode to complete the rows; the filler self-check reported 0 DRC violations and `checkFiller` found 0 gaps, finalizing the row-based layout. Figure 10 shows the final routed layout as displayed in the Encounter physical view.

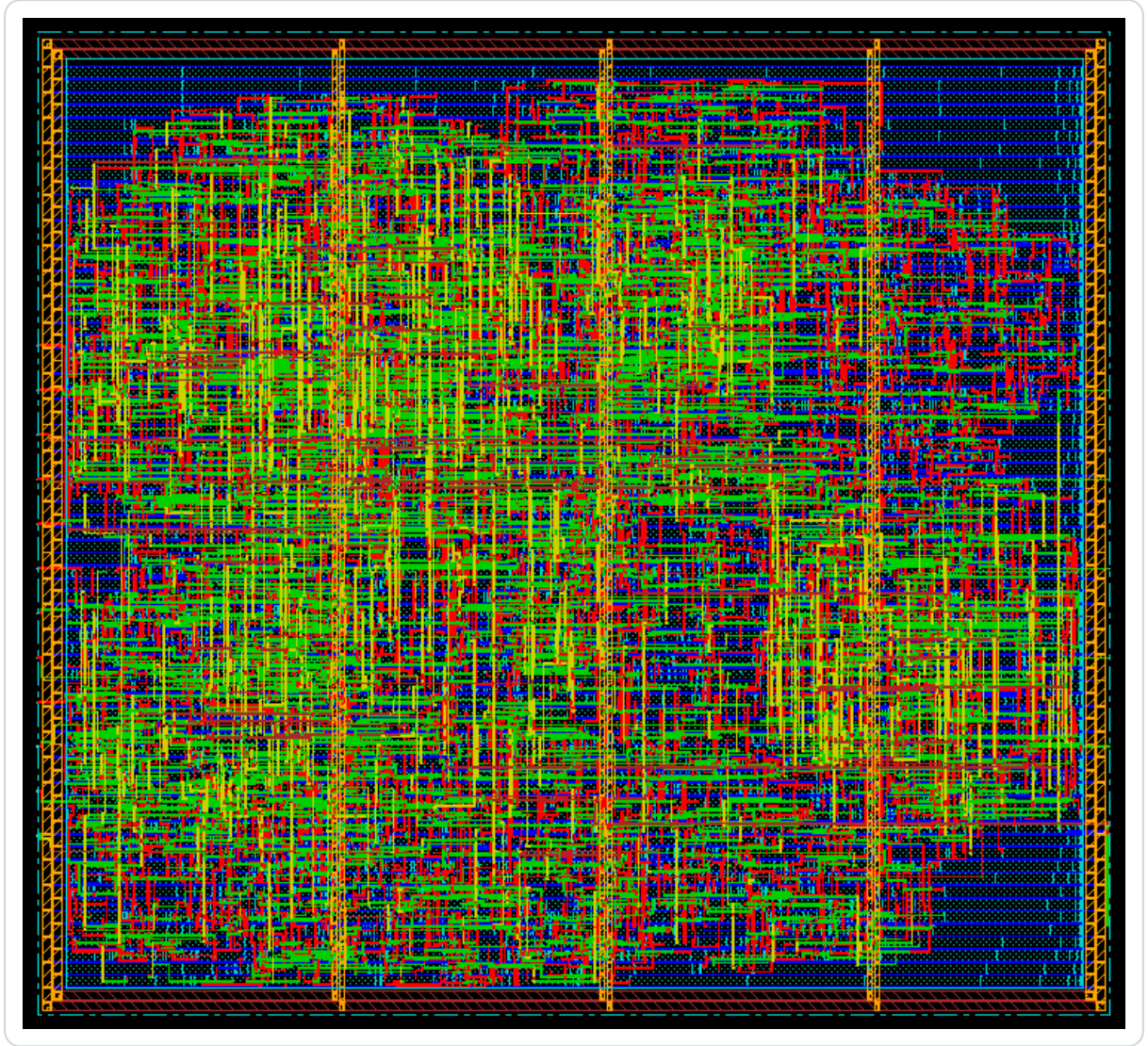


Figure 10: SYS_TOP final routed layout — physical view captured from the Encounter database (standard-cell rows with clock tree and multi-layer signal routing; die $240.47 \times 220.47 \mu\text{m}$).

12.6 Physical Verification and Sign-off

With routing and chip finish complete, the final database was checked with Encounter’s built-in verifiers and static timing analysis. The sign-off results are collected in Table 13 and summarized below:

- **Geometry (DRC):** `verifyGeometry -noMinArea` reported zero violations in every category (cells, same-net, wiring, antenna, shorts, and overlaps); the check was run both before and after filler insertion.
- **Connectivity:** `verifyConnectivity -type all` reported no problems or warnings across the routed design.
- **Process antenna:** `verifyProcessAntenna` reported 0 violations.
- **Static timing (post-route):** setup WNS +4.409 ns (TNS 0.000 ns, 0 violating paths of 1,144) and hold WNS +0.015 ns (TNS 0.000 ns, 0 violating), across all six analysis views.

- **Design rule violations:** maximum capacitance, maximum transition, and maximum fanout checks all report zero violations.
- **Power estimate:** 1.212 mW total at the `ss_1p08v_125c` corner with $VDD = 1.08\text{ V}$ and primary input activity 0.2; the estimate is vectorless and includes scan-clock activity (`dft_clk` contributes 48.39% of the total).
- **Formal equivalence:** the formal equivalence of the post-route netlist is covered in Section 13; in summary, verification SUCCEEDED with 361 passing compare points and 0 failing.

Table 13: Post-Route Sign-off Results for `SYS_TOP`.

Metric	Result	Condition / Note
Setup timing	+4.409 ns / 0.000 ns / 0	WNS / TNS / violating paths; 1,144 paths across all six views
Hold timing	+0.015 ns / 0.000 ns / 0	WNS / TNS / violating paths; post-filler re-run
Design rule violations	max_cap 0, max_tran 0, max_fanout 0	real and total counts, post-route
Clock skew (routed)	159 / 124.7 / 13.5 ps (setup); 94.3 / 52.4 / 11.6 ps (hold)	scan / UART_CLK / REF_CLK; requirement 200 ps
Routed wirelength	70,072 μm total	dominated by METAL2 (29.68 mm) and METAL3 (26.31 mm)
Vias	16,708 total; 32.6% multi-cut	single-cut 11,261; via and wire optimization applied
Density	60.30% before fill; 100.000% after fill	placement check vs. post-filler reporting
Geometry (DRC)	0 violations (all classes)	<code>verifyGeometry -noMinArea</code> ; run pre- and post-filler
Connectivity	no problems or warnings	<code>verifyConnectivity -type all</code>
Antenna	0 violations	<code>verifyProcessAntenna</code>
Power (post-route estimate)	1.212 mW total	<code>ss_1p08v_125c</code> , $VDD = 1.08\text{ V}$, input activity 0.2; vectorless; includes scan-clock activity (<code>dft_clk</code> 48.39%)
Cells / nets	2,160 leaf cells + 3,145 fillers; 2,276 nets	fillers are physical-only; 5,305 instances in GDSII
Formal equivalence	SUCCEEDED — 361 / 361 passing, 0 failing	reference RTL vs. post-route netlist; details in Section 13

12.7 GDSII Export and Deliverables

The signed-off database was exported for hand-off: GDSII via `streamOut` (units 2000 DBU, all layers), a version-3.0 SDF back-annotation file via `delayCal -sdf`, extracted parasitics via `rcOut -spf`, gate-level netlists via `saveNetlist` (both plain and with power/ground connections), and the saved Encounter database (`SYS_TOP_Final_Chip.enc`). The exported files are listed in Table 14.

The SDF (version 3.0) and the SPF export are provided for downstream static timing and parasitic analysis of the routed netlist. The stream-out completed successfully; layer-mapping diagnostics emitted during export are retained as a follow-up item for downstream reference-deck checking. The layout views captured from the saved Encounter database are presented earlier in this section (Figures 9 and 10).

Table 14: Exported Implementation Deliverables and File Sizes.

File	Contents	Size
SYS_TOP.gds	GDSII stream-out (2000 DBU, full design)	260,978 B
SYS_TOP.sdf	SDF version 3.0 timing back-annotation	2.99 MB
SYS_TOP.spf	Extracted parasitic model (<code>rcOut -spf</code>)	6.61 MB
SYS_TOP.v	Post-route gate-level netlist	250,239 B
SYS_TOP_pg.v	Netlist with power/ground connections	306,635 B

12.8 Gate-Level Simulation (Post-Route SDF)

As a final functional check on the implemented netlist, the exported post-route gate-level netlist and its version-3.0 SDF file (Table 14) were re-simulated with the same self-checking regression testbench used at the RTL stage (Table 9), compiled against the TSMC 130 nm standard-cell simulation models. The run was executed in QuestaSim 10.7c with the SDF applied in `-sdfmax` mode, i.e. at the maximum (slow) delays corresponding to 1.08 V and 125 °C as recorded in the SDF header — the setup-critical condition for timing sign-off.

SDF back-annotation completed successfully and the full directed regression ran to completion: all checks passed, terminating with the **ALL TESTS PASSED** banner and the simulator’s **[SIMULATION SUCCESSFUL]** status after ≈ 3.2 ms of simulated system operation (≈ 17 s of wall-clock simulation time). The simulator’s timing checks flagged three setup-time events, all on the first stage of the **DATA_SYNC** synchronizer (`multi_flops_reg[0]`); these are the expected consequence of sampling the asynchronous `bus_enable` input close to a clock edge and are resolved by the two-stage synchronizer by design — no functional check failed. The remaining console output consists of SDF-to-model mapping notices (missing `specify` coverage in the cell simulation models, and post-PnR instance/port name mapping), which are tool-flow messages rather than design issues.

The gate-level pass completes the verification stack for the delivered netlist: it is formally equivalent to the RTL (Section 13), meets the post-route static timing sign-off (Table 13), and reproduces the complete directed regression functionally correct under real back-annotated delays. The full-hierarchy switching-activity trace (VCD) captured during this run is reused as the activity source for the VCD-annotated power analysis of Section 14.1.

13 Synopsys Formality Formal Equivalence Verification

Formal Equivalence Checking (LEC) was executed using **Synopsys Formality** [6] across three sign-off stages (post-synthesis, post-DFT, and post-place-and-route). Figure 8 (Section 12) illustrates the verification phases within the complete backend flow. The physical implementation itself is described in Section 12.

All three formal runs converged with **100% Comparison Success**:

- **Post-Synthesis** (RTL vs. post-synthesis netlist): **379** compare points (375 DFF + 3 ports + 1 latch); verification **SUCCEEDED**.
- **Post-DFT** (pre-DFT vs. post-DFT netlist; scan chain insertion, mission mode): **361** compare points (358 DFF + 3 ports); verification **SUCCEEDED**.
- **Post-PnR** (post-DFT vs. post-PnR netlist; SI/SO scan ports excluded): **361** compare points (358 DFF + 3 ports); verification **SUCCEEDED**.

In total, **0** failing and **0** unmatched compare points; two reference registers (`deserializer_counter_reg[3]`, `data_reg[7]`) were reported as unread (optimized in the implementation

netlist) and excluded via standard unread/constant handling.

14 Power Optimization and Dynamic Power Profiling

The ALU datapath is idle during most UART-dominated execution cycles, so its clock branch is gated by a latch-based Integrated Clock Gating (ICG) cell. The `CLK_GATE` module instantiates the TSMC 130nm library ICG cell (`TLATNCAX2M`) in synthesis mode; its enable (`Gate_EN`) is asserted by the master FSM only during ALU execution, and the active-low latch captures the enable in the clock-low phase, keeping the gated clock glitch-free.

Power was profiled in the Synopsys DC/Power Compiler flow at 100 MHz; the conditions for each estimate are noted below.

- **Synthesis estimate** (`Synthesis/reports/power.rpt`; SS/1.08 V/125 °C corner; default propagated activity — primary inputs unannotated, low-effort vectorless simulation): total ≈ 0.484 mW (internal 0.412 mW, switching 54.7 μ W, leakage 18.2 μ W).
- **Post-scan estimate** (`DFT/reports/power.rpt`): total ≈ 0.166 mW (leakage 15.6 μ W).
- **Post-route estimate** (`System_pnr/pnr/report/power.rpt`; primary-input activity 0.2): total 1.212 mW at 1.08 V (internal 70.3%, switching 28.1%, leakage 1.6% ≈ 19.9 μ W). This figure includes the DFT scan-clock domain ($\approx 48\%$ of the total power) and therefore overestimates functional-mode power; the VCD-annotated analysis below quantifies the functional-mode figure.

All estimates above use default or vectorless activity where noted. To complement them, a switching-activity-annotated analysis was performed on the post-route netlist using the activity trace captured during the gate-level regression, as described next.

14.1 VCD-Annotated Power Analysis (PrimeTime PX)

A time-based power analysis was executed with Synopsys PrimeTime PX (O-2018.06-SP1) on the post-route netlist, driven by the VCD switching-activity trace captured during the gate-level regression of Section 12.8. Because the activity is taken from the actual functional run — including the real clock-gating behavior of the ALU and the genuine `REF_CLK/UART_CLK` activity over ≈ 3.2 ms — the result characterizes normal operation rather than assumed activity factors. The session resolves the typical-corner library (TT: 1.2 V, 25 °C).

Table 15: PrimeTime PX VCD-annotated power summary for the post-route netlist (functional workload).

Power Component	Value	Share of Total
Net switching power	0.67 μ W	5.7%
Cell internal power	3.21 μ W	27.4%
Cell leakage power	7.84 μ W	66.9%
Total power	11.72 μW	100%

Two run conditions are noted for completeness: clock definitions were not applied in this first analysis session (clock-network and register power are therefore not separately attributed), and PrimeTime reports partial VCD coverage with its power-may-be-underestimated warning; the reported total is consequently treated as a *lower bound* for functional-mode power, with a clock- and SDF-loaded rerun at fuller coverage identified as a refinement step. Even as a lower bound, the result quantitatively confirms the low-power behavior of the design under normal operation: with the scan clock idle and a UART-dominated workload, the reported average power is leakage-dominated ($\approx 67\%$) and lies roughly two orders of magnitude below the scan-inclusive, slow-corner post-route estimate of the item above.

15 Summary of Implementation Results and Sign-off Metrics

Table 16 consolidates the complete architectural and physical sign-off metrics achieved across the ASIC implementation flow.

Table 16: Consolidated SoC Architectural & ASIC Implementation Sign-off Metrics.

Metric Category	Parameter	Achieved Sign-off Value
Architecture	Reference Clock Frequency (REF_CLK)	100.0 MHz ($T_{\text{clk}} = 10.0$ ns)
	UART Peripheral Frequency (UART_CLK)	3.6864 MHz ($T_{\text{clk}} = 271.27$ ns)
	Maximum Serial Baud Rate	115,200 Baud (Programmable via REG3)
Physical (130nm)	Total Active Standard Cell Area	27,225.31 μm^2
	Total Standard Cell Count	2,275 Cells
	Sequential Flip-Flop Count	380 Scan Flip-Flops (4 chains $\times$ 95)
Physical Implementation (Encounter)	Die Size	240.47 $\times$ 220.47 μm
	Placement Density	56.8%
	Routed Wire / Vias	70,072 μm / 16,708 Vias
	Post-route Timing (Setup / Hold)	+4.409 ns / +0.015 ns (MET)
	GDSII Stream-out	260,978 B
Timing (SS / 125°C)	Setup Slack (Worst Negative Slack)	0.00 ns (MET at 100 MHz)
	Hold Slack	+0.43 ns (MET)
Power (100 MHz)	Post-route estimate	1.212 mW (SS/1.08 V, activity 0.2; incl. scan clock)
	VCD-annotated (functional, PrimeTime PX)	11.72 μW (TT corner; leakage-dominated lower bound)
DFT & Testability	Scan Chain Cells	380 (4 $\times$ 95)
	Test Coverage	99.66% (Synopsys DFT Compiler)
	Fault Class Summary	17,305 DT / 333 non-detected of 17,638
Verification Closure	SpyGlass CDC & Lint Sign-off	0 errors (lint: 6 waived STARC05 warnings; CDC: 0 unsynchronized crossings; 1 Gray-coded convergence — check PASSED)
	Formality Formal Equivalence	379 / 361 / 361 Compare Points (post-synth / post-DFT / post-PnR)
	Gate-Level Simulation (post-route SDF)	All tests passed — complete directed regression with back-annotated delays (Section 12.8)

16 Conclusion and Future Work

This final project successfully demonstrated the microarchitecture, directed self-checking RTL verification, static CDC sign-off, logic synthesis, scan testability insertion, and formal verification of a low-power multi-clock digital communication SoC subsystem.

By strategically deploying enable-handshake data synchronization, Gray-coded dual-clock asynchronous FIFO buffering, and dual-domain asynchronous reset synchronizers, the architecture eliminates metastability and multi-bit data incoherency across asynchronous clock boundaries, while a latch-based Integrated Clock Gating cell provides design-level clock gating for dynamic power reduction. The project delivered the complete flow from RTL through verification, SpyGlass sign-off, logic synthesis, DFT, place-and-route, in-tool physical sign-off checks, post-route static timing analysis, post-route gate-level simulation with SDF back-annotation, and a VCD-annotated power analysis, with formal equivalence confirmed across all three sign-off stages and culminating in a signed-off core-level GDSII layout. The design closed timing at 100 MHz under worst-case corner conditions with positive post-route slack (worst setup +4.409 ns, worst hold +0.015 ns) and zero violating paths; scan insertion achieved 99.66% test coverage with four scan chains, and all in-tool checks are clean.

16.1 Limitations

The following scope boundaries apply to the delivered results:

- **Verification scope.** Verification consists of the executed directed regression: reset behavior, register-file write/readback across parity configurations, and TX frame validation; extended ALU, error-injection, and FIFO-stress scenarios are pending.
- **Gate-level simulation.** The post-route gate-level regression (slow-corner SDF) passes the complete directed test suite; minimum-corner (hold) gate-level runs and extended stress scenarios are not included.
- **Physical verification.** Physical checks are tool-level (Encounter verify commands); no foundry-grade DRC/LVS decks were run.
- **Power analysis.** The vectorless estimates are complemented by the VCD-annotated PrimeTime PX result (Section 14.1), which reflects the functional workload and is treated as a lower bound.
- **Layout scope.** The GDSII stream is core-level; no pad ring or full-chip assembly is included.

16.2 Future Work

The following items are identified as future work:

- Full pad-ring and full-chip assembly.
- Foundry-grade physical verification (Calibre-class DRC/LVS) and a re-stream of the GDSII with a corrected layer map and complete cell geometry.
- Clock-attributed, full-coverage refinement of the VCD-annotated power analysis.
- Minimum-corner (hold) gate-level regression with the post-route SDF, plus gate-level stress and error-injection scenarios.
- IR-drop and electromigration (EM) analysis.
- An optional APB/AXI bus wrapper for SoC integration.

References

- [1] C. E. Cummings, “Simulation and Synthesis Techniques for Asynchronous FIFO Design,” *SNUG (Synopsys Users Group) San Jose*, 2002.
- [2] C. E. Cummings, “Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog,” *SNUG Boston*, 2008.
- [3] IEEE Standard for Verilog Hardware Description Language, *IEEE Std 1364-2005*, IEEE Computer Society, 2006.
- [4] Synopsys, Inc., *Design Compiler User Guide: Logic Synthesis and Optimization*, Version O-2018.06-SP1, Mountain View, CA, 2018.
- [5] Synopsys, Inc., *DFT Compiler User Guide: Scan Insertion and ATPG*, Version O-2018.06-SP1, Mountain View, CA, 2018.
- [6] Synopsys, Inc., *Formality User Guide: Equivalence Checking*, Version L-2016.03-SP1, Mountain View, CA, 2016.
- [7] Synopsys, Inc., *SpyGlass CDC Methodology and Rules Reference Manual*, Version vL-2016.06, Mountain View, CA, 2016.
- [8] Cadence Design Systems, *First Encounter / SoC Encounter User Guide*, v8.10, 2008.
- [9] C. E. Cummings, “Asynchronous & Synchronous Reset Design Techniques,” *SNUG Boston*, 2003.

Appendix A — Implementation Flow Scripts

This appendix presents representative excerpts from the implementation flow scripts that produced the results reported in this document; the complete scripts, reports, and log files accompany the project repository (Appendix C). Long option lists in the excerpts are abbreviated for readability.

The first excerpt shows the timing constraints generated by `write_sdc`, covering the three primary clocks — `ref_clk` (100 MHz), `uart_clk` (3.6864 MHz), and `dft_clk` (1 MHz scan clock) — together with clock setup/hold uncertainty and representative input/output delays.

```
1 # SYS_TOP.sdc (excerpt)
2 create_clock [get_ports REF_CLK] -name ref_clk -period 10 -waveform {0 5}
3 set_clock_uncertainty -setup 0.2 [get_clocks ref_clk]
4 set_clock_uncertainty -hold 0.1 [get_clocks ref_clk]
5 create_clock [get_ports UART_CLK] -name uart_clk -period 271.267 -waveform {0
  135.634}
6 set_clock_uncertainty -setup 0.2 [get_clocks uart_clk]
7 set_clock_uncertainty -hold 0.1 [get_clocks uart_clk]
8 create_clock [get_ports scan_clk] -name dft_clk -period 1000 -waveform {0 500}
9 set_clock_uncertainty -setup 0.2 [get_clocks dft_clk]
10 set_input_delay -clock uart_clk 54.2535 [get_ports UART_RX_IN]
11 set_output_delay -clock uart_clk 54.2535 [get_ports UART_TX_0]
12 set_input_delay -clock dft_clk 200 [get_ports {SI[3]}]
13 set_output_delay -clock dft_clk 200 [get_ports {S0[3]}]
```

The second excerpt condenses the Cadence First Encounter place-and-route command sequence (`encounter.cmd`), from floorplanning and power planning through clock-tree synthesis, routing, physical verification, chip finishing, and GDSII stream-out.

```
1 # encounter.cmd (condensed excerpt)
2 floorPlan -d 240.47 220.47 6.0 6.0 6.0 6.0
3 addRing -around core -layer_bottom METAL5 -layer_top METAL5 -layer_left METAL6 -
  layer_right METAL6 -nets {VSS VDD} ...
4 addStripe -set_to_set_distance 60 -layer METAL6 -width 1 -nets {VSS VDD} ...
5 sroute -connect { blockPin padPin padRing corePin floatingStripe } -nets { VSS VDD }
  ...
6 placeDesign -inPlaceOpt -prePlaceOpt
7 clockDesign -genSpecOnly Clock.ctstch
8 clockDesign -specFile Clock.ctstch -outDir clock_report -fixedInstBeforeCTS
9 optDesign -postCTS -hold
10 routeDesign -globalDetail -viaOpt -wireOpt
11 verifyGeometry -noMinArea
12 verifyConnectivity -type all -noAntenna -error 1000 -warning 50
13 verifyProcessAntenna -reportfile SYS_TOP.antenna.rpt -leffile SYS_TOP.antenna.lef -
  error 1000
14 addFiller -cell {FILL1M FILL2M FILL4M FILL8M FILL16M FILL32M FILL64M} -prefix FILLER -
  markFixed
15 saveNetlist export/SYS_TOP.v
16 streamOut export/SYS_TOP.gds -mapFile ./import/gds2InLayer.map -libName DesignLib -
  stripes 1 -units 2000 -mode ALL
```

The third excerpt shows the gate-level simulation flow used for the post-route SDF regression of Section 12.8: compilation of the cell models, the post-route netlist, and the testbench (with VCD activity dumping enabled), followed by maximum-delay SDF back-annotated simulation of the complete regression.

```
1 # run.do (excerpt) -- post-route gate-level simulation with SDF
2 vlog -sv std_cells/tsmc13_m.v
3 vlog -sv sdc_sdf_netlist/SYS_TOP.v
4 vlog -sv +define+DUMP_VCD +define+IVERILOG SYSTEM_TOP_tb.v
5 vsim -sdfmax /SYSTEM_TOP_tb/DUT=sdc_sdf_netlist/SYS_TOP.sdf -sdfnoerror +no_notifier
  work.SYSTEM_TOP_tb
6 run -all
```

Appendix B — Regression Transcript (Excerpt)

The excerpt below is quoted from the archived regression transcript (`do_files/transcript`) produced by ModelSim Intel FPGA Edition (vlog 2020.1 compiler) on October 3, 2026 during the directed self-checking regression. It lists the executed cases with representative self-check PASS lines, followed by the final success banner and exit summary.

```
1 # CASE 1 : PARITY OFF
2 # PASS REG[4] after write : 55
3 # PASS TX received byte : 55
4 # PASS STOP bit = 1
5 # CASE 2 : SECOND DATA PATTERN
6 # PASS REG[5] after write : a5
7 # PASS TX received byte : a5
8 # PASS STOP bit = 1
9 # CASE 3 : EVEN PARITY
10 # PASS REG[6] after write : 3c
11 # PASS PARITY bit = 00000000
12 # PASS STOP bit = 1
13 # CASE 4 : ODD PARITY
14 # PASS REG[7] after write : 96
15 # PASS PARITY bit = 00000001
16 # PASS STOP bit = 1
17 # =====
18 #           ALL TESTS PASSED
19 # =====
20 # Errors: 0, Warnings: 0
```

Appendix C — Reproducibility and Tool Versions

Table 17 summarizes the EDA tools, versions, and technology library used to produce the results reported in this document.

Activity	Tool & Version
Simulation	QuestaSim/ModelSim (Intel FPGA Edition v2020.1); open-source Icarus Verilog for continuous-integration regression
Gate-level simulation	QuestaSim 10.7c (SDF back-annotated, post-route netlist)
Power analysis	Synopsys PrimeTime PX O-2018.06-SP1 (VCD-annotated, time-based)
Lint / CDC	Synopsys SpyGlass vL-2016.06 (GuideWare/latest/block/rtl_handoff methodology)
Synthesis / DFT	Synopsys Design Compiler O-2018.06-SP1 (DFT Compiler, Power Compiler)
Formal verification	Synopsys Formality L-2016.03-SP1
Place and route	Cadence First Encounter v08.10-p004 (NanoRoute)
Technology library	Metro TSMC 0.13 μ m c1013g RVT standard-cell library

Table 17: Tool versions and technology library used to produce the reported results.

All flow scripts, constraint files, reports, testbenches, and transcripts are archived in the project repository: <https://github.com/ahmedbadwy77/Low-Power-Multi-Clock-Digital-Communication-System>.